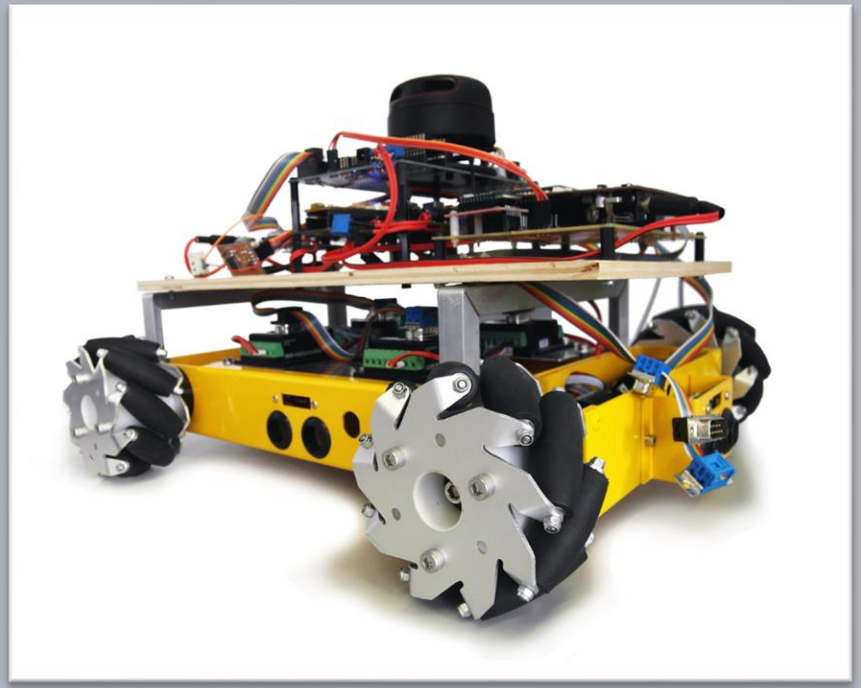
Advanced Embedded Architectures and Applications

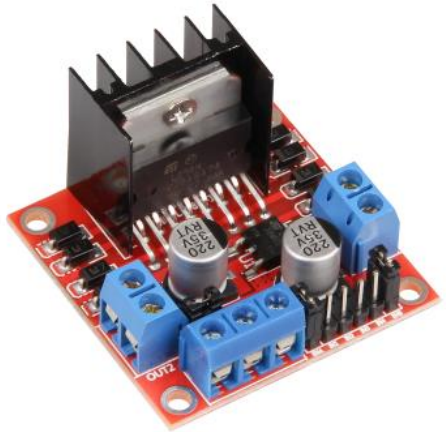
Engine Complex Device Driver

Prof. Dr.-Ing. Peter Fromm

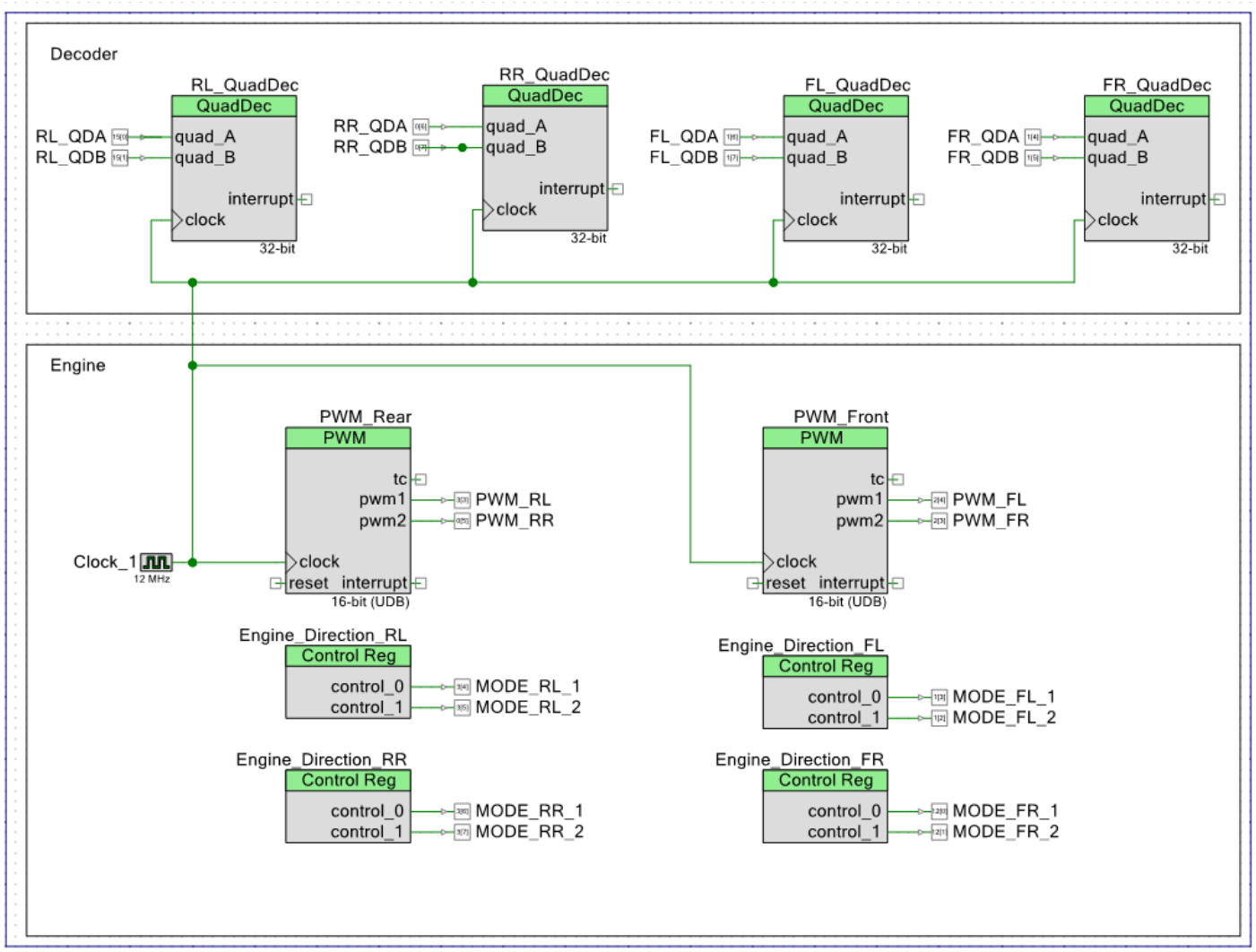


Content

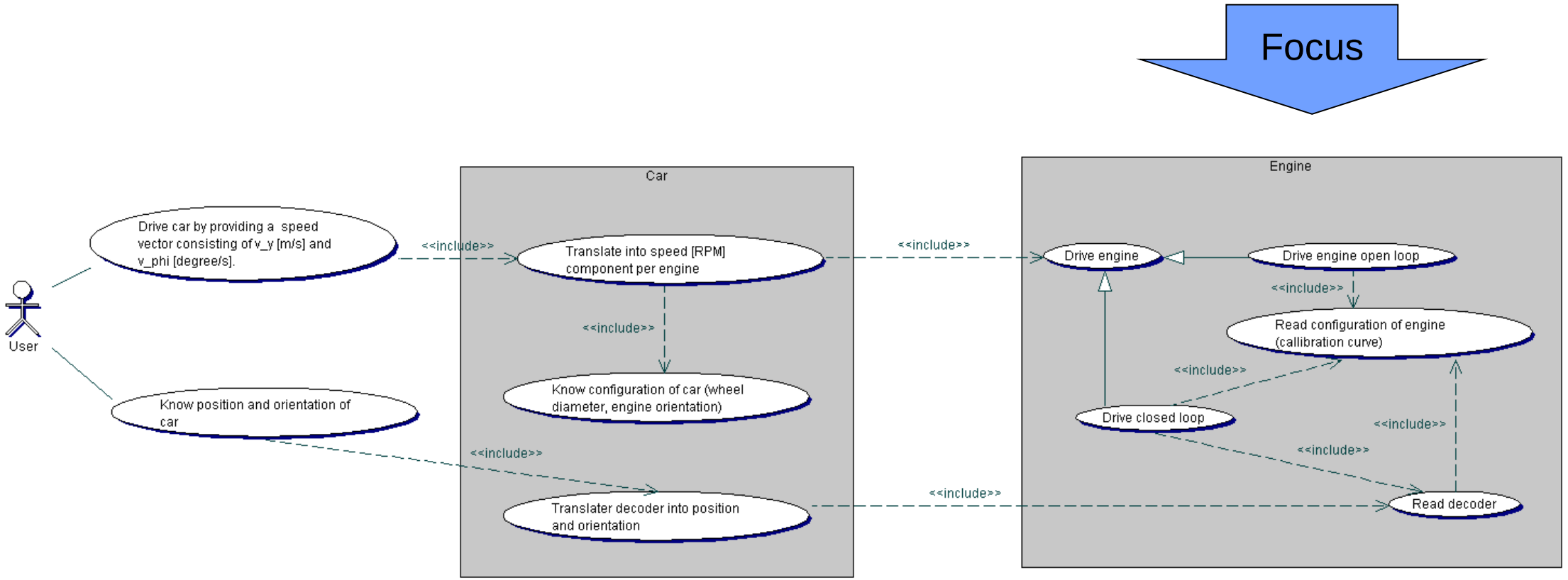
- What we have and what we need
- Driving the Engine using PWM
- Quadriple Decoder
- Analyzing the engine behavior
- Open Loop Control
  - Linear approach
  - Calibration curve
- Driving all 4 engines
- Closed loop control
- Controller Tuning



What we have


Decoder
RL_QuadDec
QuadDec
RL_QDA
RL_QDB
quad_A
quad_B
interrupt
clock
32-bit
RR_QuadDec
QuadDec
RR_QDA
RR_QDB
quad_A
quad_B
interrupt
clock
32-bit
FL_QuadDec
QuadDec
FL_QDA
FL_QDB
quad_A
quad_B
interrupt
clock
32-bit
FR_QuadDec
QuadDec
FR_QDA
FR_QDB
quad_A
quad_B
interrupt
clock
32-bit
Engine
PWM_Rear
PWM
tc
pwm1
pwm2
PWM_RL
PWM_RR
clock
reset
interrupt
16-bit (UDB)
Clock_1
12 MHz
PWM_Front
PWM
tc
pwm1
pwm2
PWM_FL
PWM_FR
clock
reset
interrupt
16-bit (UDB)
Engine_Direction_RL
Control Reg
control_0
control_1
MODE_RL_1
MODE_RL_2
Engine_Direction_FL
Control Reg
control_0
control_1
MODE_FL_1
MODE_FL_2
Engine_Direction_RR
Control Reg
control_0
control_1
MODE_RR_1
MODE_RR_2
Engine_Direction_FR
Control Reg
control_0
control_1
MODE_FR_1
MODE_FR_2

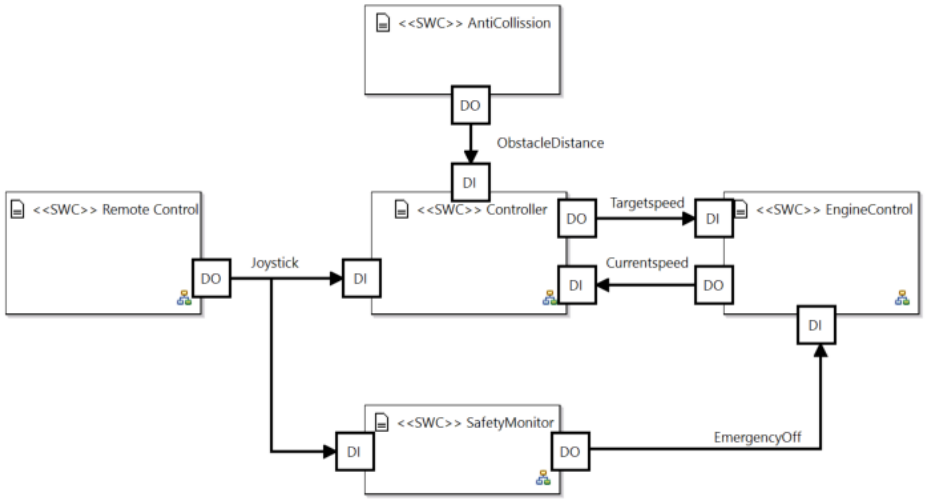
And what we need

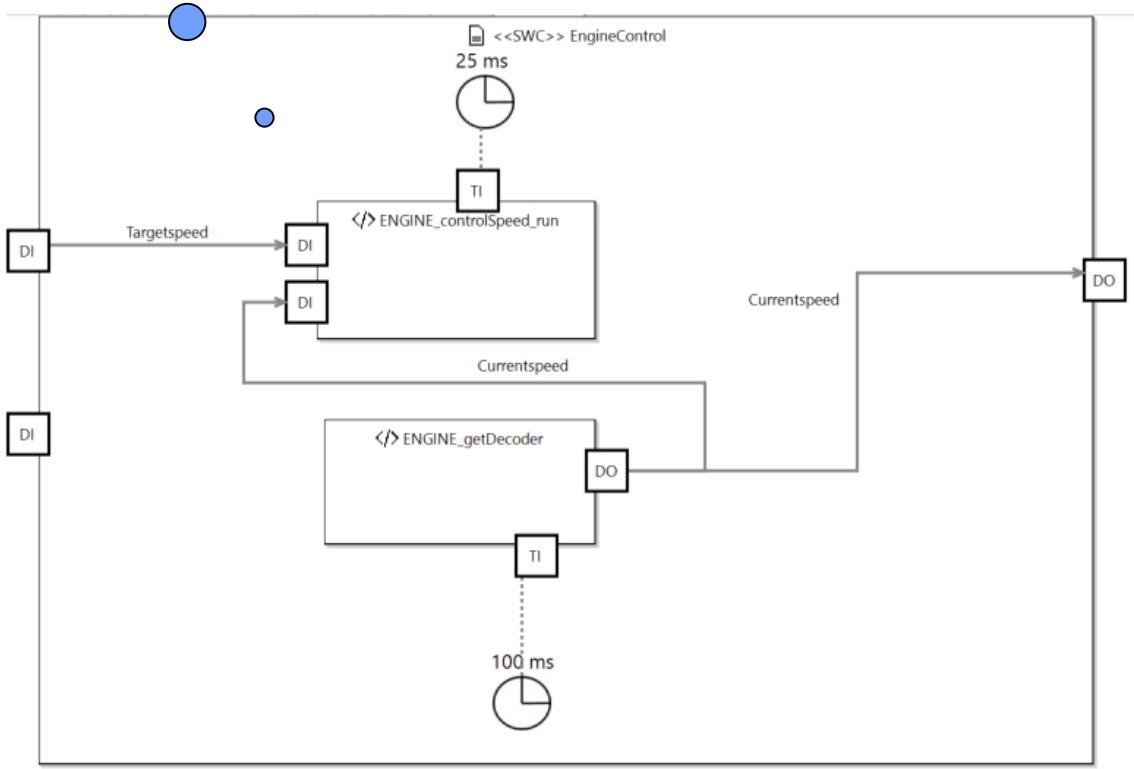


Design Options

What will be part of the runnable, what will be part of the driver?

1 or 2 runnables?





Speed

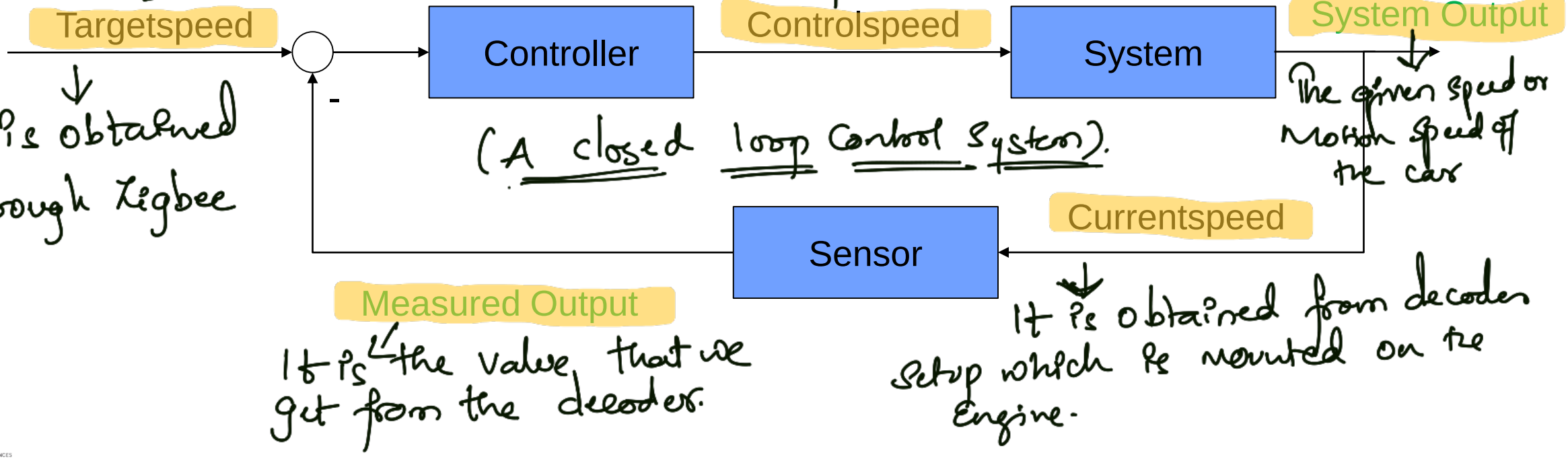
Control Error happen because, we suddenly increase the speed and the engine is physically not able to follow this target speed that quickly, the reason might be Engine is blocking a bit due to the friction, bumpy ground etc.

- Targetspeed – Speed requested by the user
- Controlspeed – Speed value given to the engine driver (H-bridge)
- Currentspeed – Speed measured at the engine

The delta between the targetspeed and the current speed is Control Error

Error

It is the combination of value which are sent to the pwm components of mode values



It is obtained through Zigbee

(A closed loop control system).

The given speed or motion speed of the car

It is obtained from decoder setup which is mounted on the Engine.

It is the value that we get from the decoder.

Let's start on MCAL level
A little bit of physics

Mechanical and electrical Power of an engine (simplified)

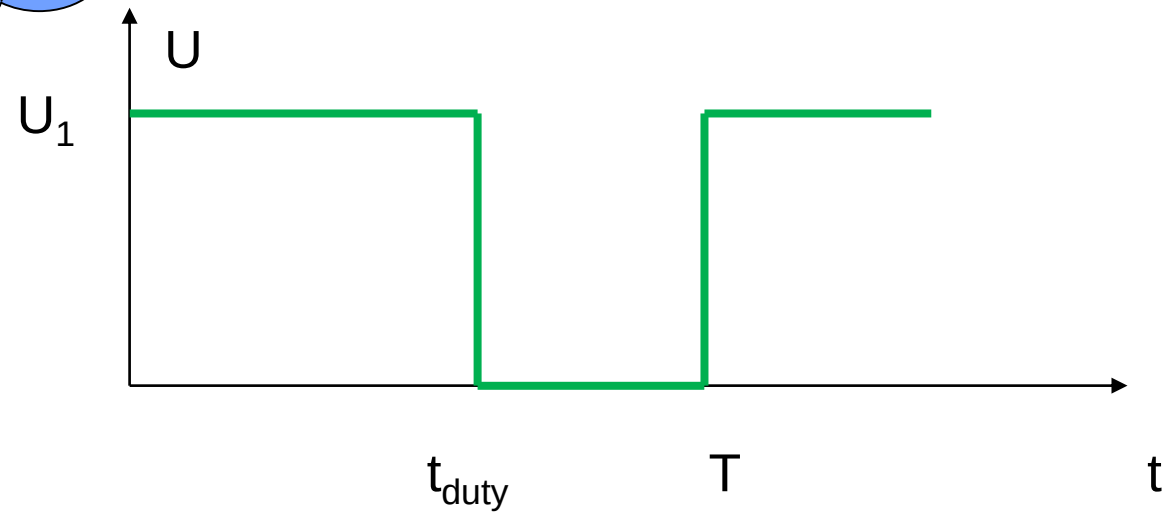
$$P_{mech} = \vec{M} \times \vec{\omega}$$

$$P_{el} = U \times I = \frac{\int_0^T U(t) \times I(t) dt}{T}$$

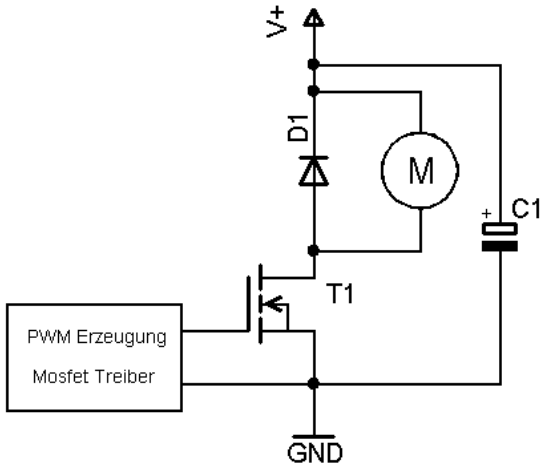
$$duty = \frac{t_{duty}}{T}$$

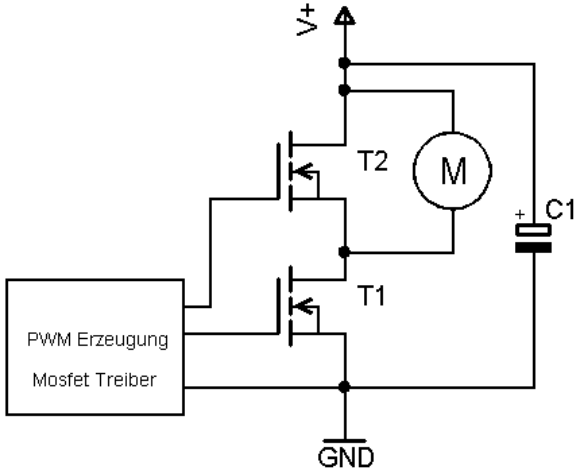
$$P_{el} = P_{mech} \rightarrow |\omega| = \frac{U_1 \times I \times duty}{M}$$

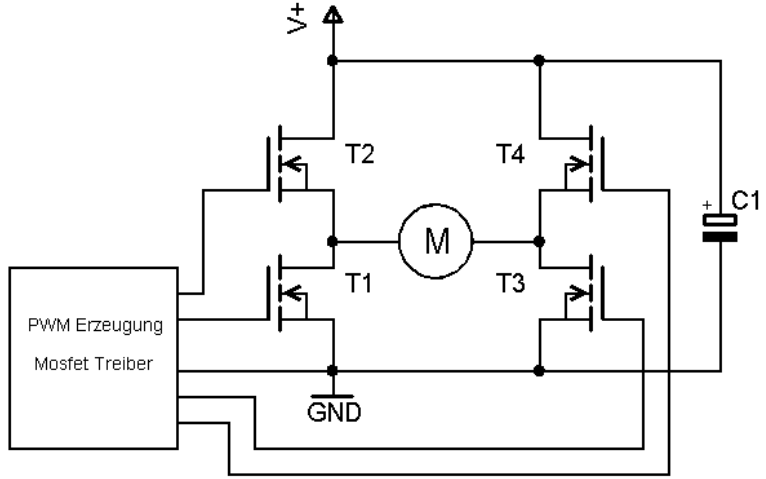
Assuming I and M const.



Schematics Power Driver





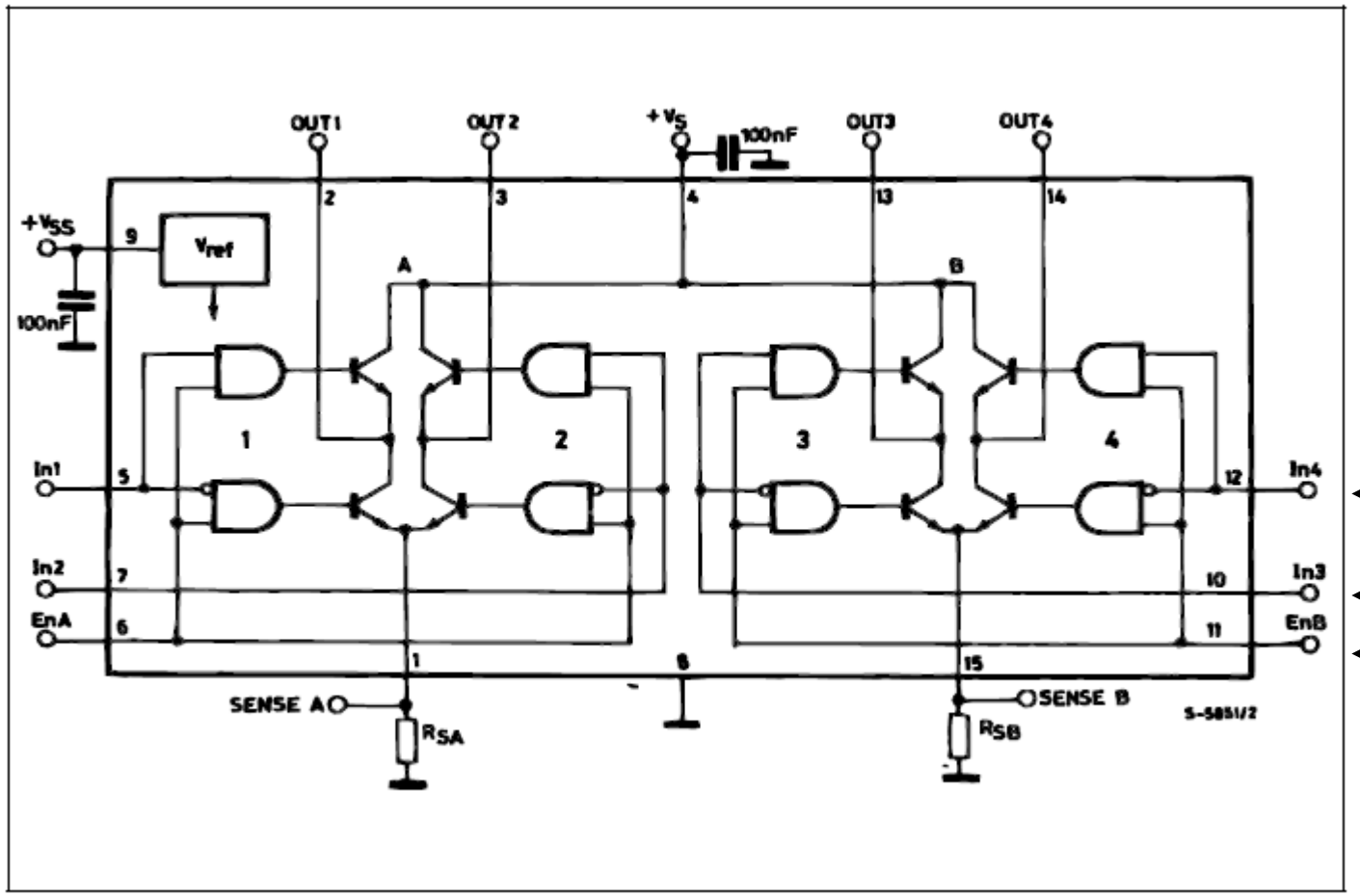


1 Quadrant Controller	2 Quadrant Controller	4 Quadrant Controller
1 direction No brake	1 direction Recuperative brake	2 directions Recuperative brake

Source: https://www.mikrocontroller.net/articles/Motoransteuerung_mit_PWM

Setting the speed
L298 DUAL FULL-BRIDGE DRIVER

BLOCK DIAGRAM



Sailing Mode

0/1

1/0

PWM

Brake Mode

PWM / 0

0 / PWM

1

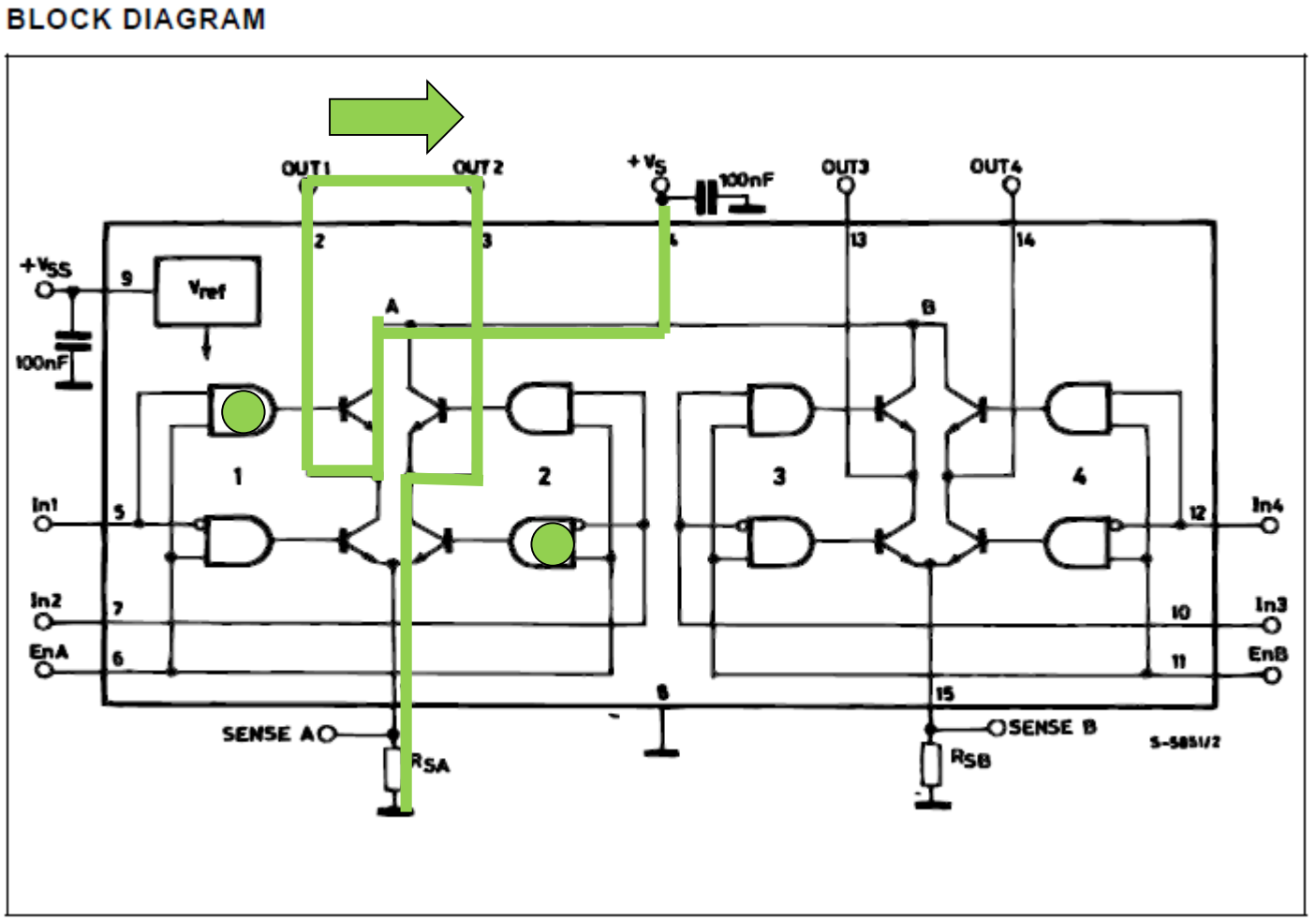
Setting the speed
L298 DUAL FULL-BRIDGE DRIVER - Forward

Sailing Mode

1

0

PWM



Setting the speed
L298 DUAL FULL-BRIDGE DRIVER - Reverse

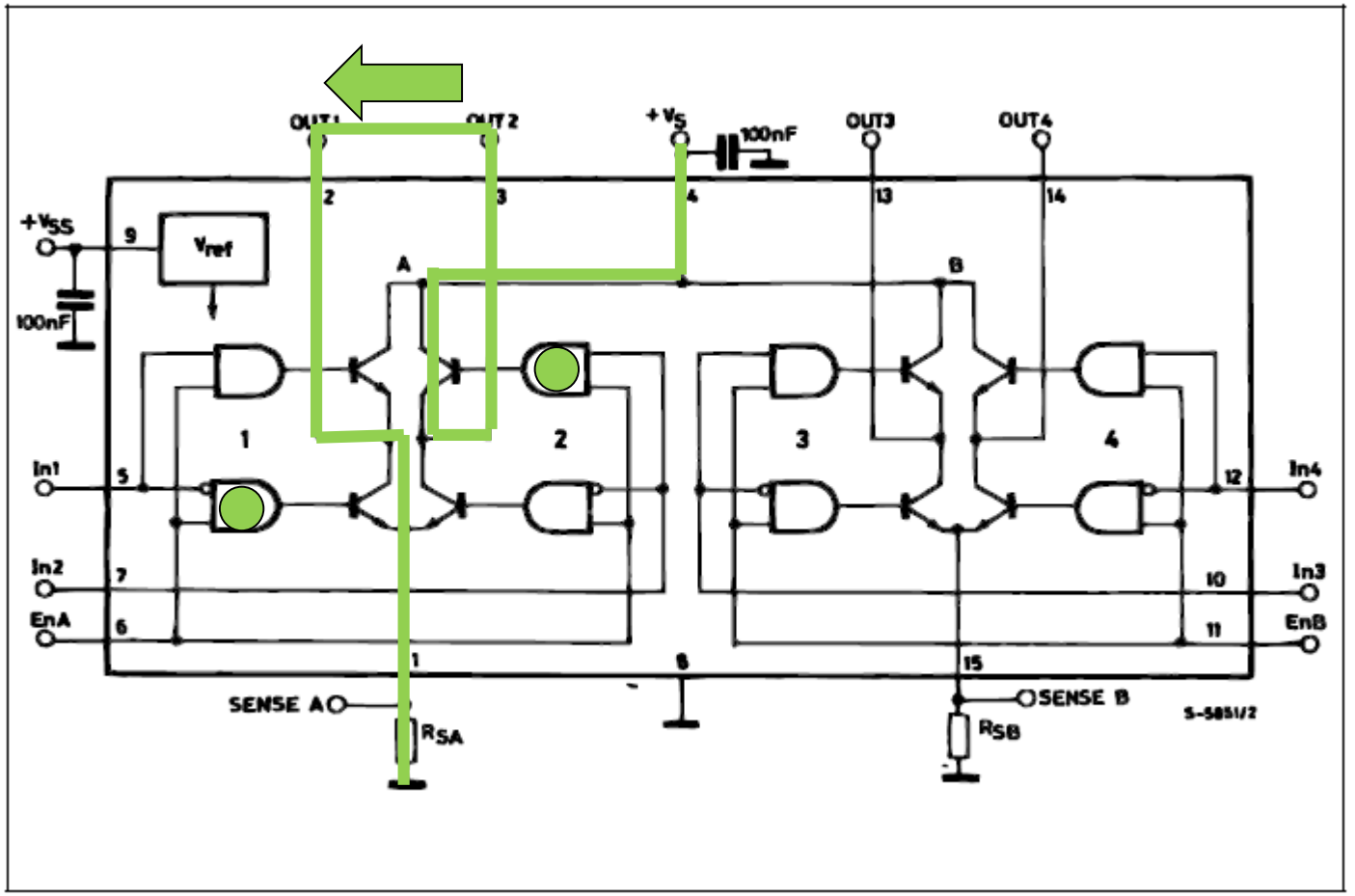
Sailing Mode

0

1

PWM

BLOCK DIAGRAM



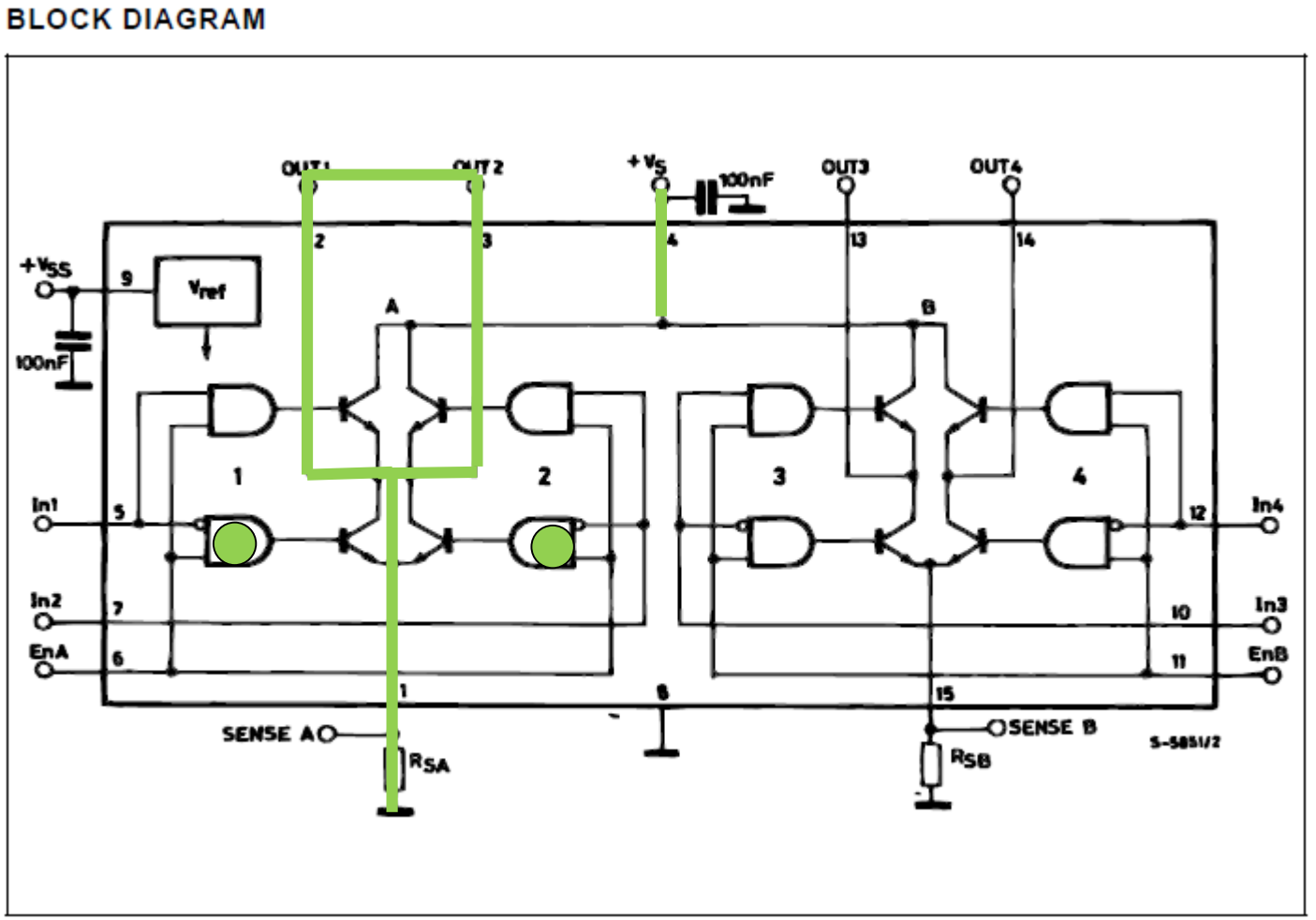
Setting the speed
L298 DUAL FULL-BRIDGE DRIVER - Brake

Sailing Mode

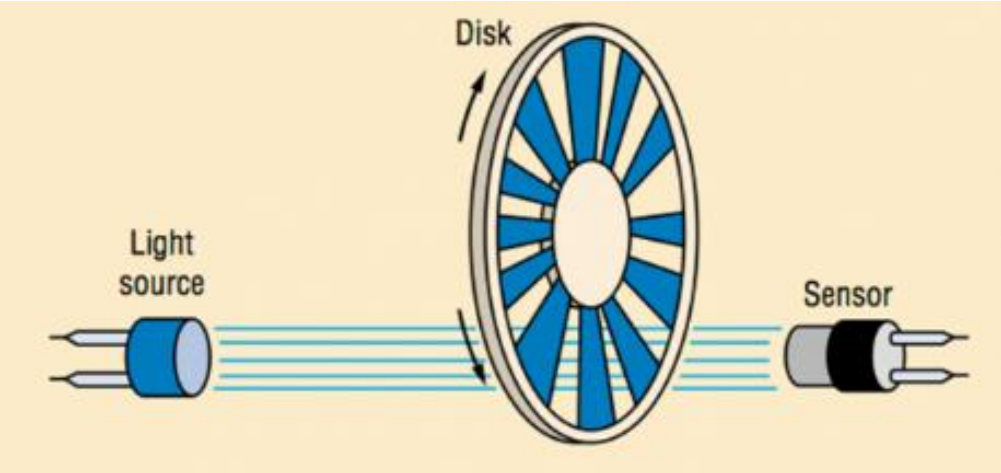
0

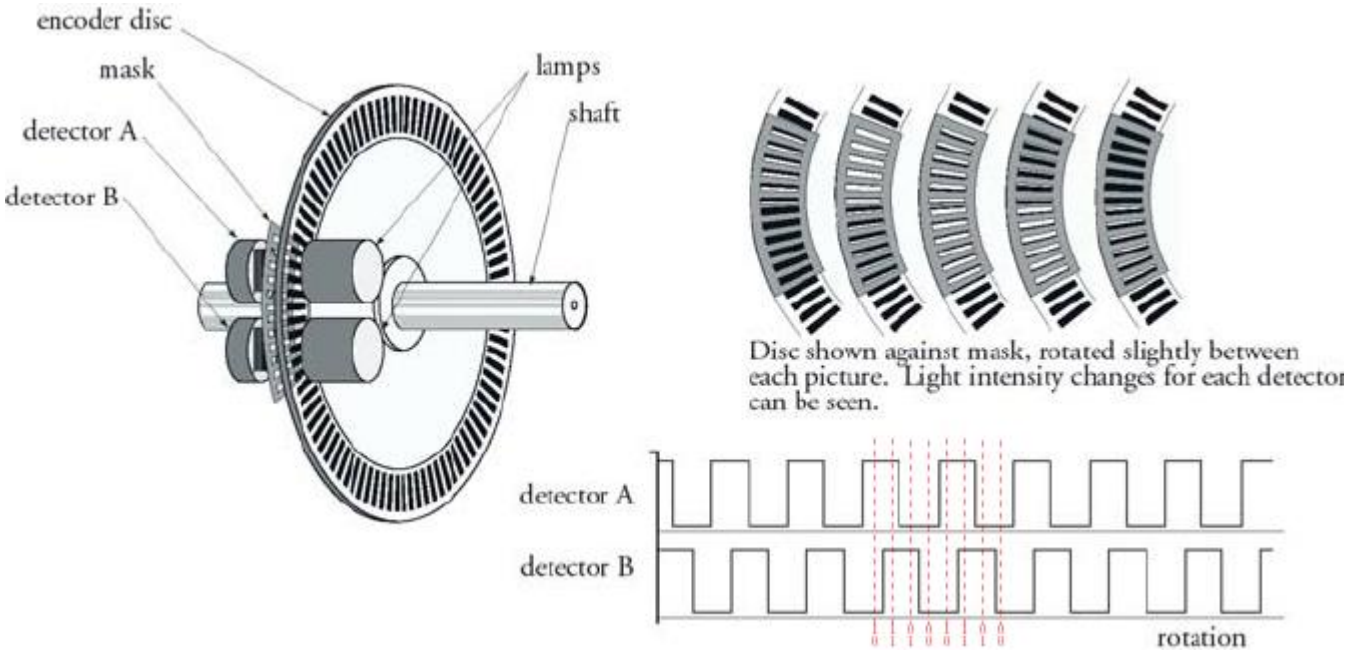
0

PWM



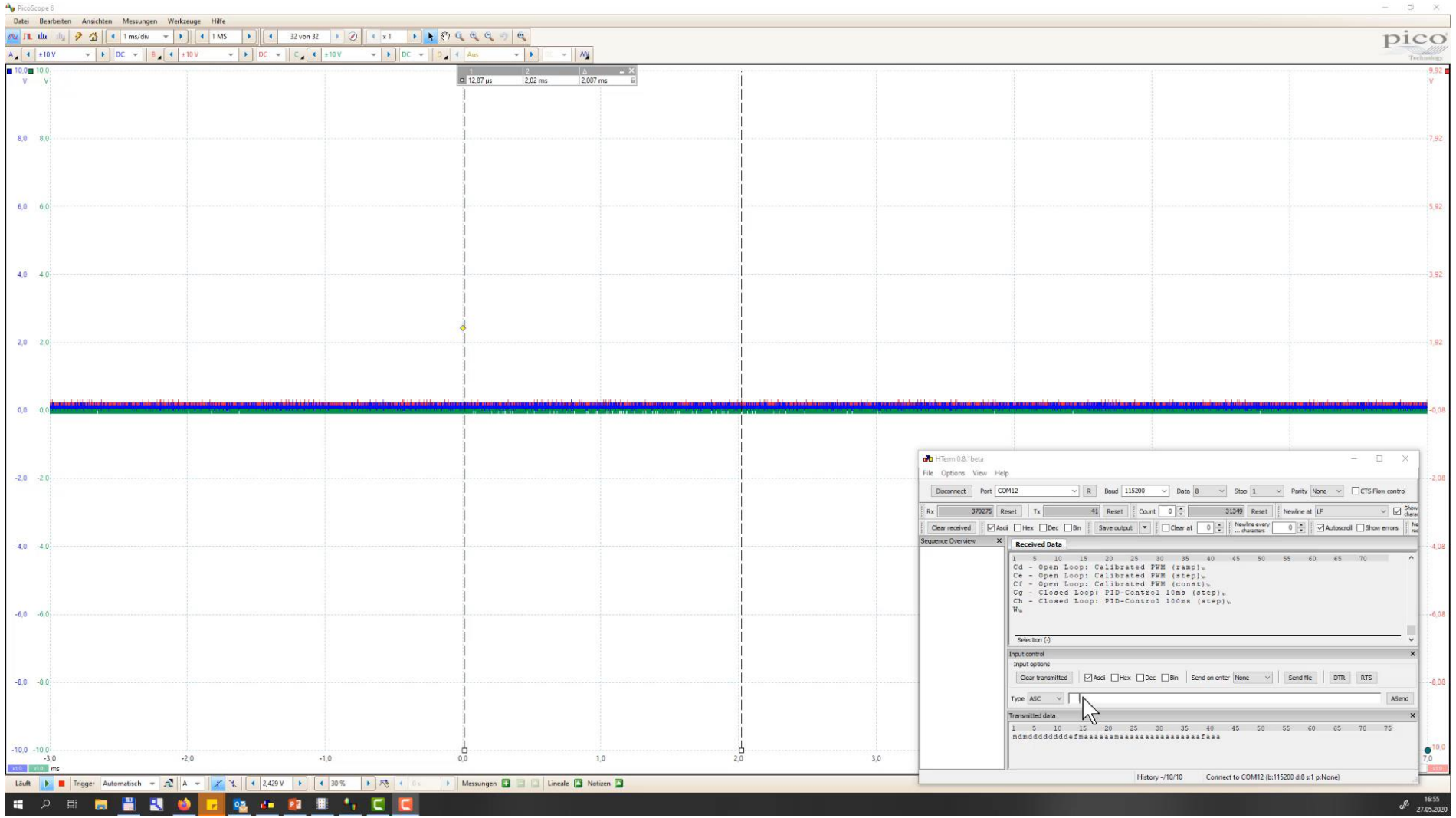
Optical Encoder



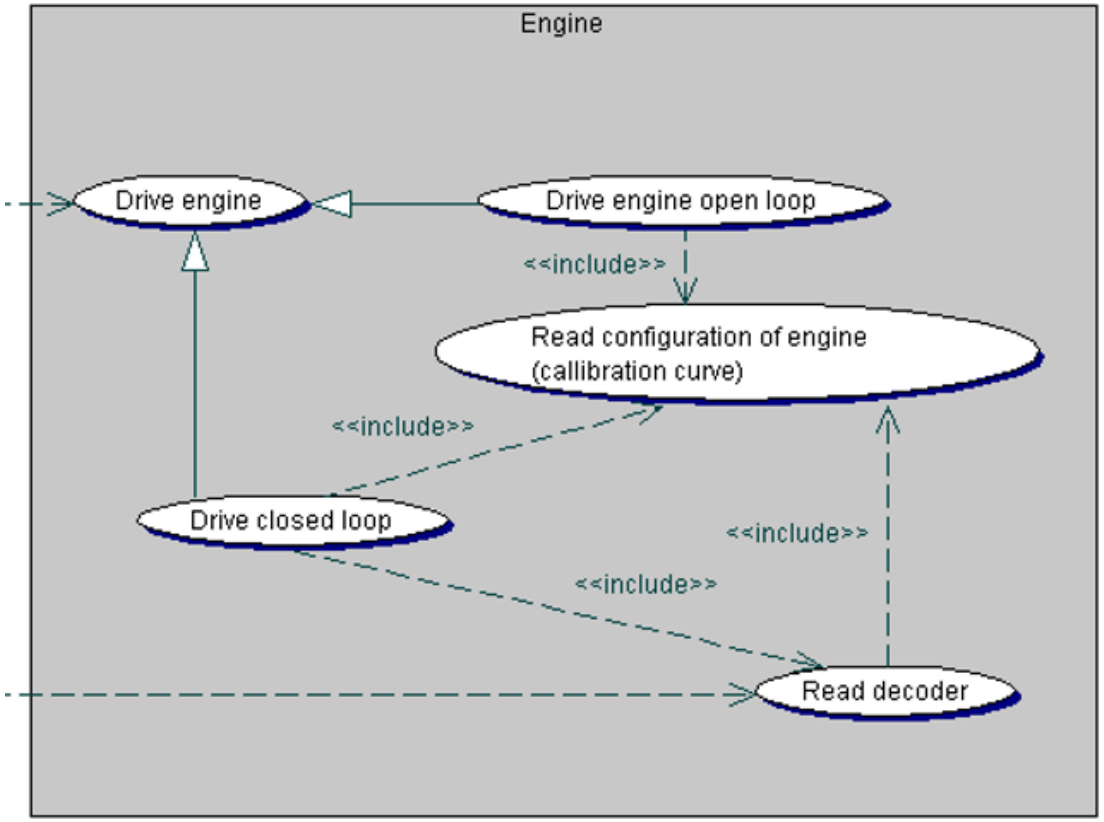


Source: https://electronics.stackexchange.com/questions/458915/how-to-make-affordable-diy-low-res-absolute-rotatory-optical-encoders-with-fiber

Getting the Speed
Quadriple Decoder



Let’s create a first design

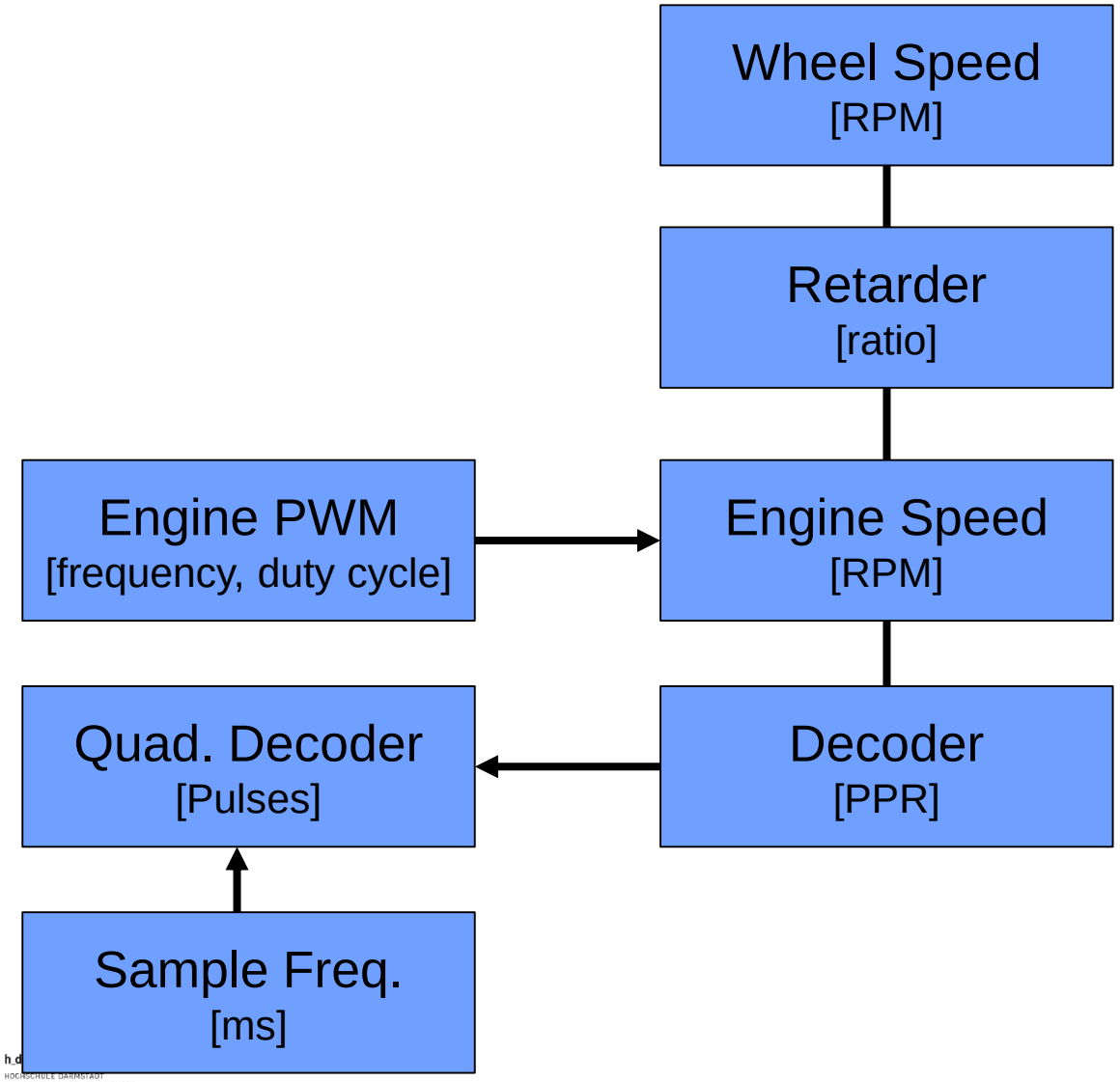


- Which API functions do we need?
- What parameters?

- Idea: The engine object.

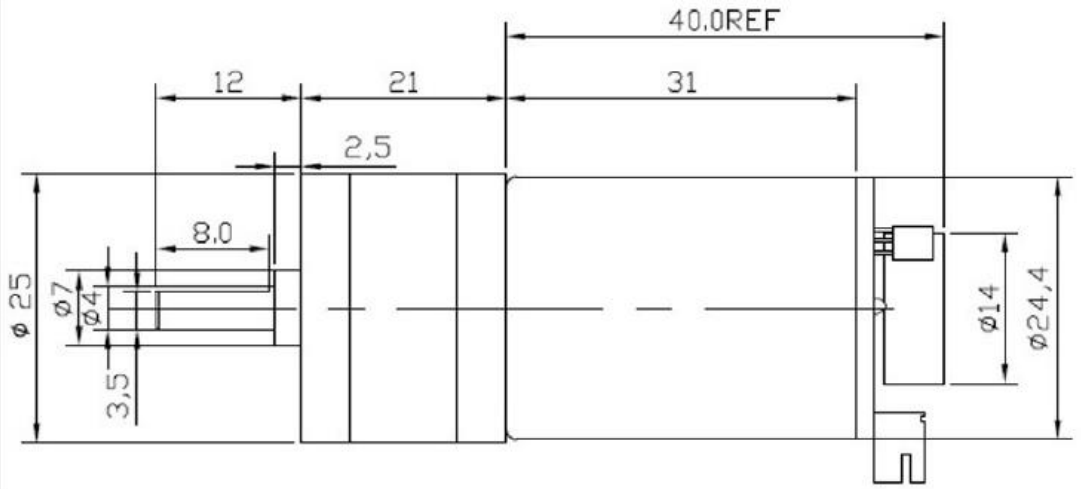
Let’s go life!

The calibration story: Physics and Units



Gear Motor parameter list	
Rated voltage	DC6.0V
No-load speed	210RPM 0.13A
Max efficiency	负载2.0kg.cm/170rpm/2.0W/0.60A
Max power	负载5.2kg.cm/110rpm/3.1W/1.10A
Stall	10kg.cm 堵死电流 3.2A
Retarder reduction ratio	1 : 34
Holzer resolution	Motor Holzer×ratio34.02=341.2PPR

Outline drawing



Test environment

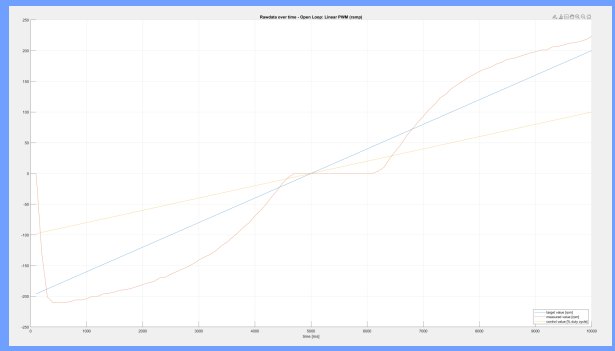
MCU

```
static const ENGINE__Testrecord_t ENGINE__testcases[] = {
//   cmd  menustring                                  initfunc                  cyclicfunc                  deinitfunc                cycletime   end time
    {'m', "Show this menu",                          0,                         ENGINE__showMenu,           0,                        0,          0},
    {'t', "Terminate any runnable",                  0,                         ENGINE__terminate,          0,                        0,          0},

    {'a', "Open Loop: Linear PWM (ramp)",            ENGINE__StartMeasurement,  ENGINE__LinearCurve,        ENGINE__EndMeasurement,   100,        10000},
    {'b', "Open Loop: Create Calibration Forward (ramp)", ENGINE__StartMeasurement, ENGINE__CreateCalibrationForward, ENGINE__EndMeasurement, 100,    40000},
    {'c', "Open Loop: Create Calibration Reverse (ramp)", ENGINE__StartMeasurement, ENGINE__CreateCalibrationBackward, ENGINE__EndMeasurement, 100,   40000},

    {'d', "Open Loop: Calibrated PWM (ramp)",        ENGINE__StartMeasurement,  ENGINE__OpenLoopRamp,       ENGINE__EndMeasurement,    100,        50000},
    {'e', "Open Loop: Calibrated PWM (step)",        ENGINE__StartMeasurement,  ENGINE__OpenLoopStep,       ENGINE__EndMeasurement,    10,         5000},
    {'f', "Open Loop: Calibrated PWM (const)",       ENGINE__StartMeasurement,  ENGINE__OpenLoopConst,      ENGINE__EndMeasurement,    100,        10000},

    {'g', "Closed Loop: PID-Control 10ms (step)",    ENGINE__StartMeasurement,  ENGINE__ClosedLoopPIDStep,  ENGINE__EndMeasurement,    10,         10000},
    {'h', "Closed Loop: PID-Control 100ms (step)",   ENGINE__StartMeasurement,  ENGINE__ClosedLoopPIDStep,  ENGINE__EndMeasurement,    100,        10000},

};
```

Speed

Testcase

Matlab



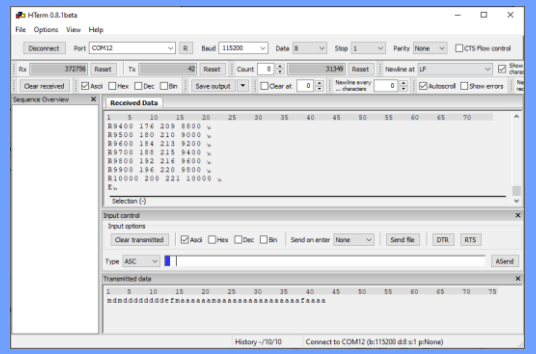
Controlspeed

Currentspeed

Speed

Testcase

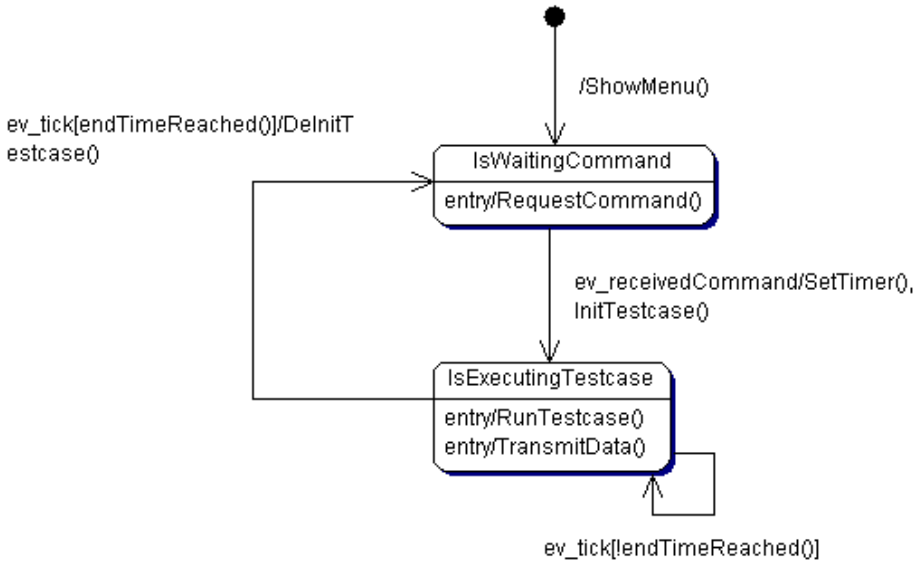
HTerm



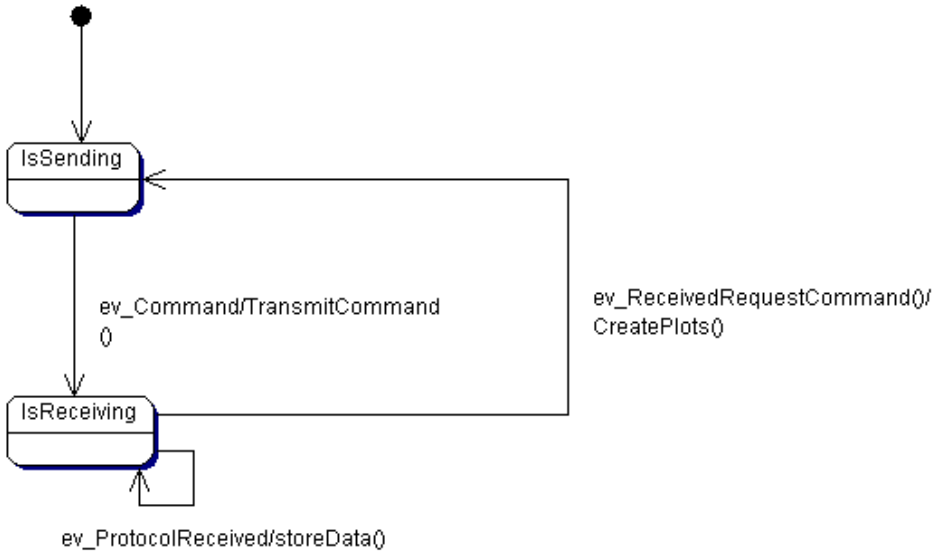
Alternative Debugging

Communication between MCU and Matlab

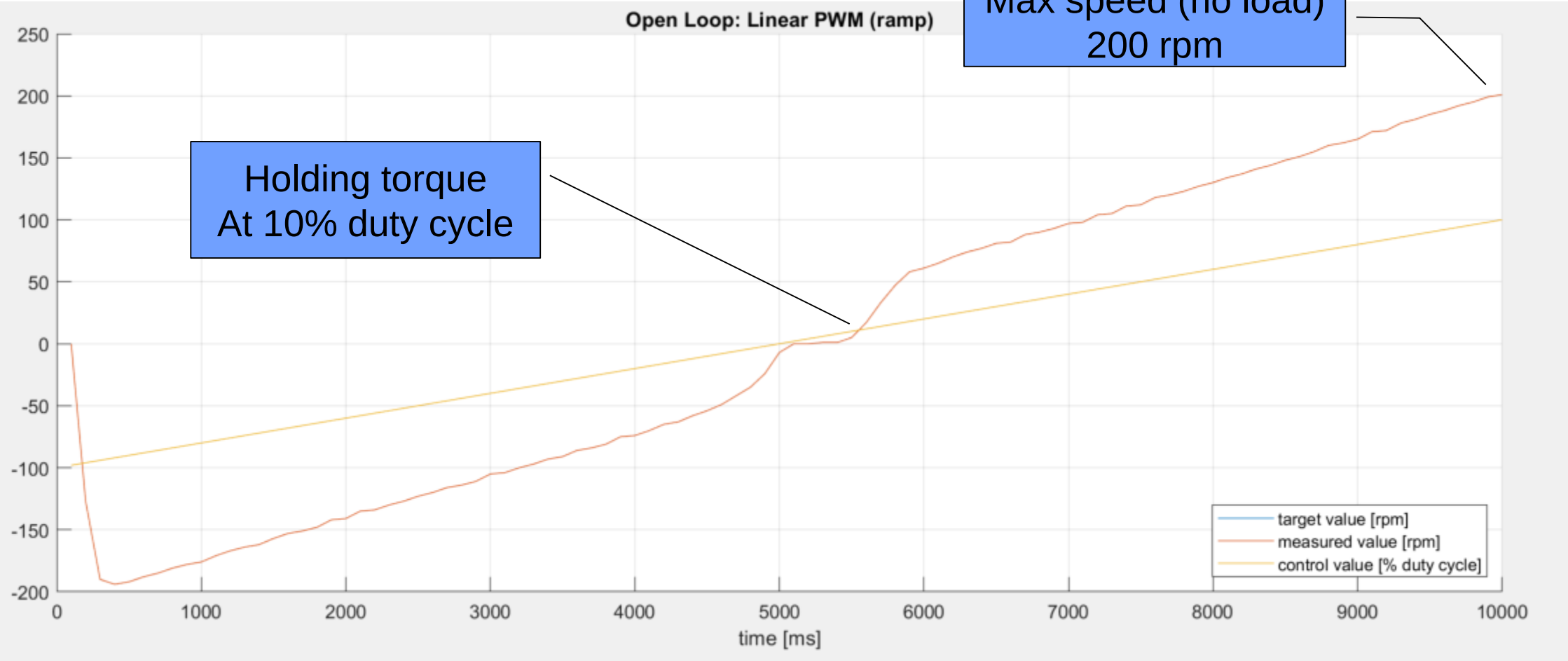
MCU



Matlab

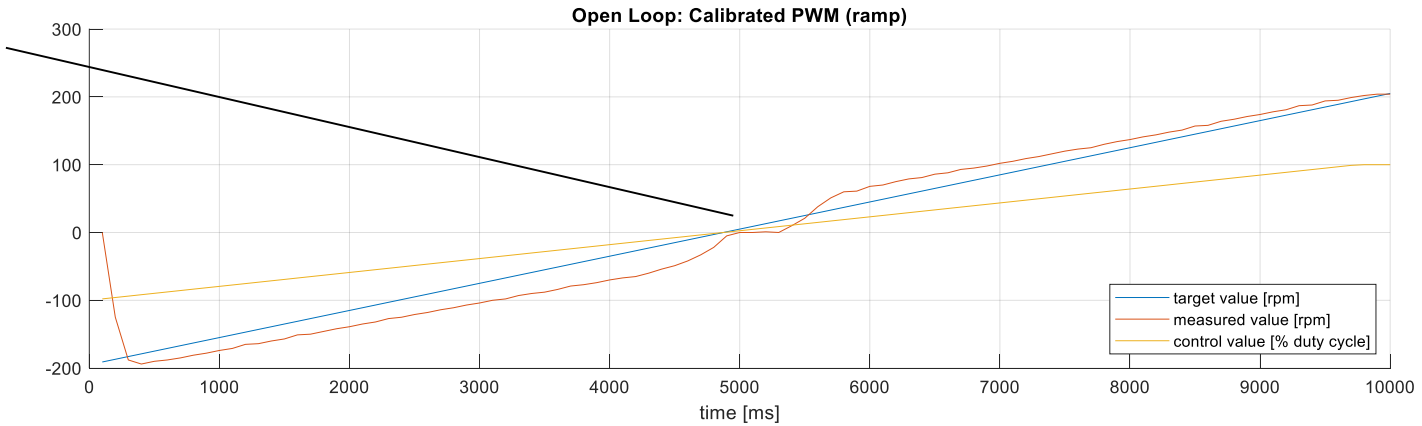


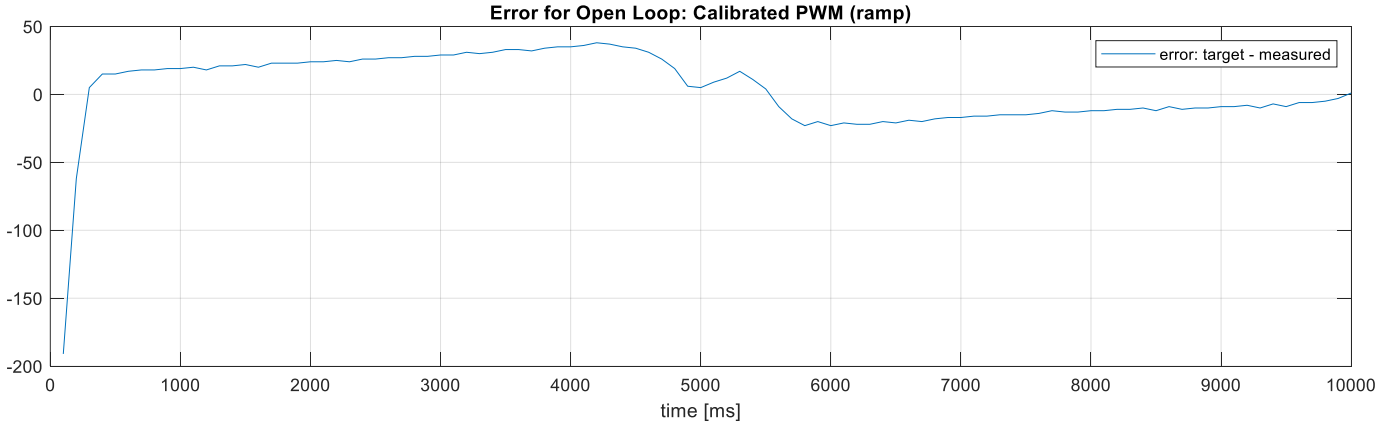
A first curve – linear increase of PWM duty cycle [100kHz] / Brake Mode



Linear approximation – not really good! Even for Brake mode

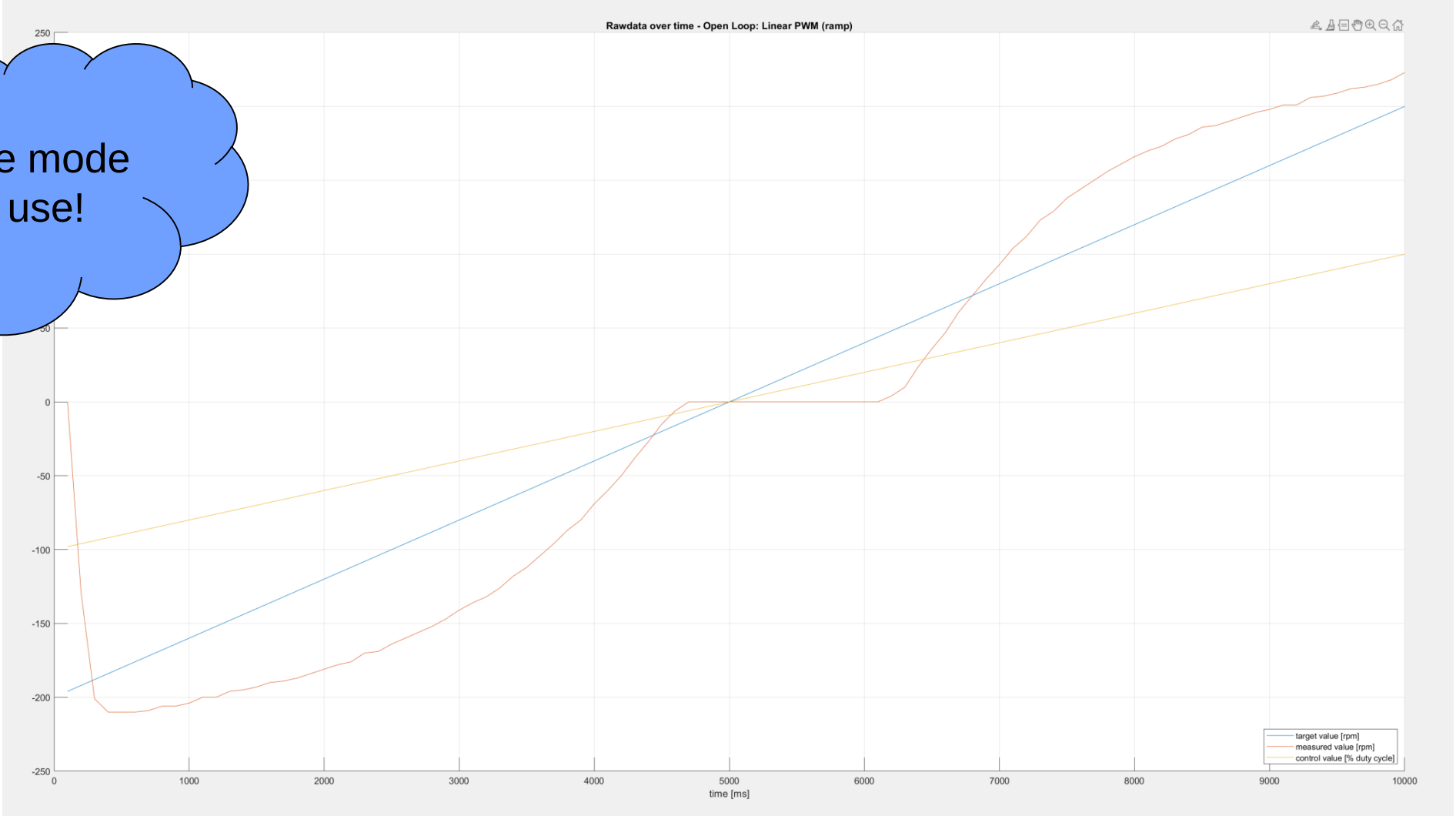
Note the hysteresis





A first curve – linear increase of PWM duty cycle [100kHz] / Sailing Mode

This is the mode we will use!



Concept of a lookup table

→ The Table below contains the { target value, Measured PWM value } The measured pwm value is nothing but result.

Targetspeed [rpm]

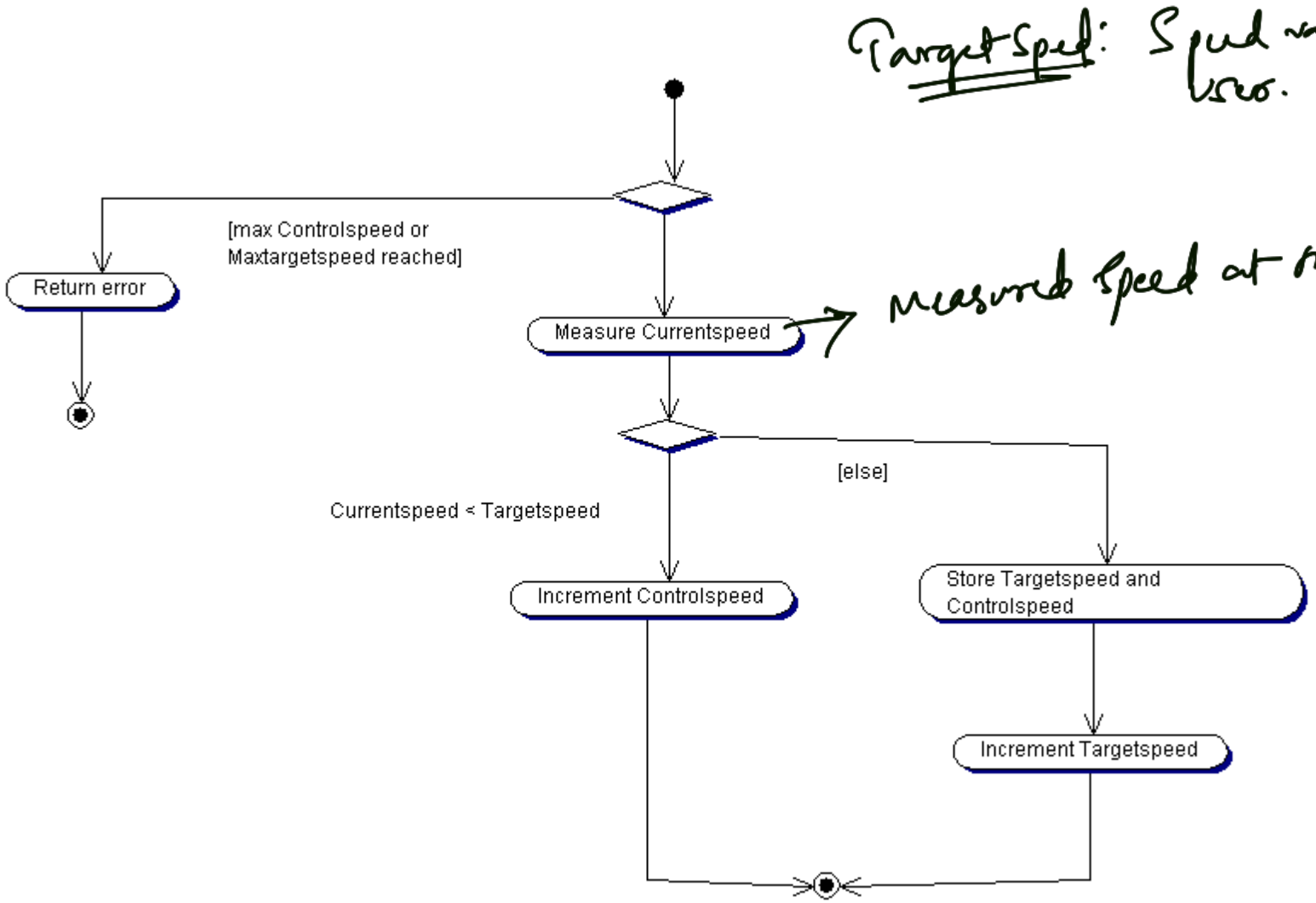
```
const ENG_calibration_t
ENG_cal_backward[] = {
  { 0 , 0 },
  { -5 , -2700 },
  { -10 , -2740 },
  { -15 , -2780 },
  { -20 , -2810 },
  { -25 , -2900 },
  { -30 , -2950 },
      …
};
```

→ Let's say we want to move with -7 RPM, we would take values -5 & -10 and do some linear approximations, to calculate a value inbetween these two, which might be around 2720

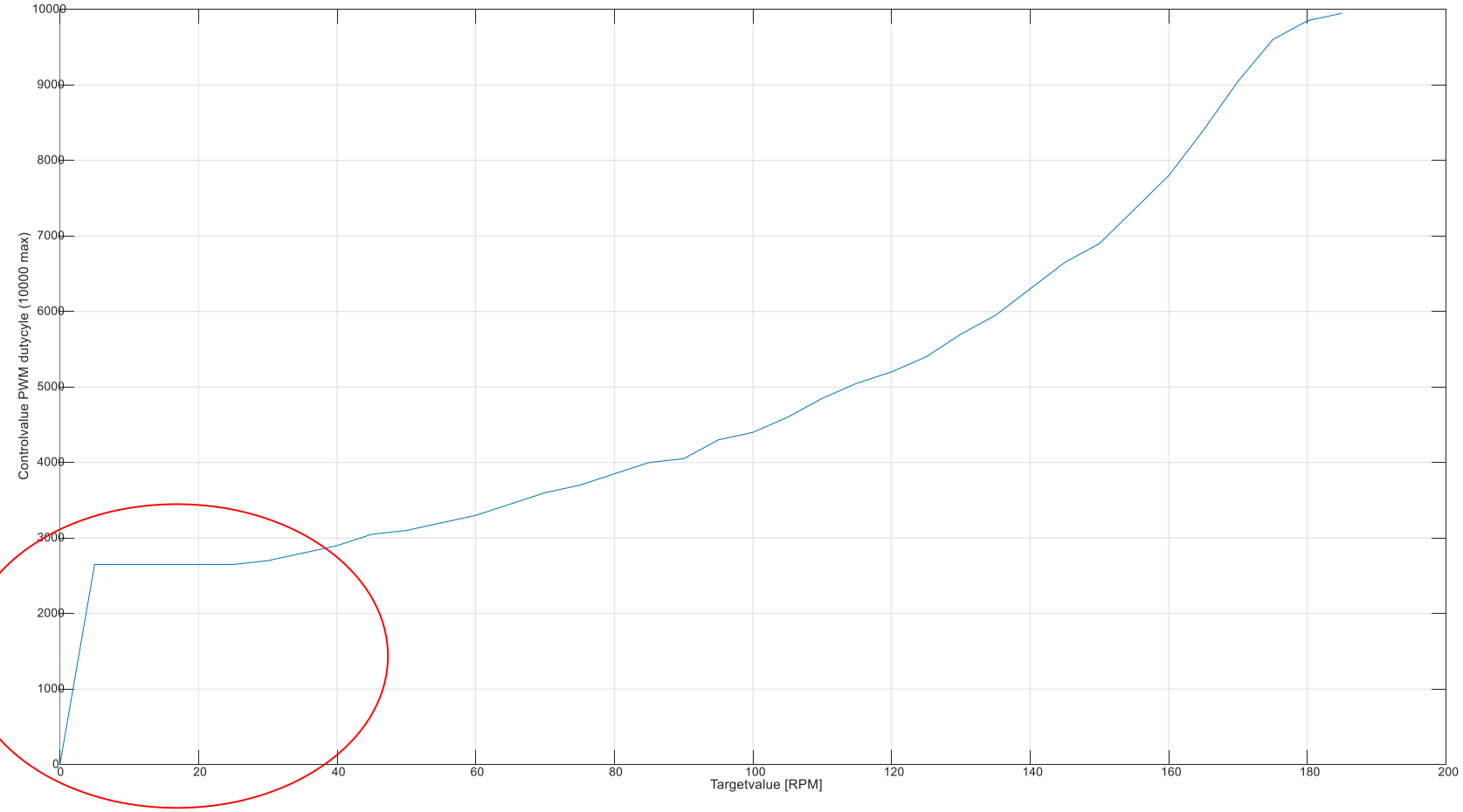
Controlspeed [dutycycle]

Then this control value will be given as duty cycle to PWM.

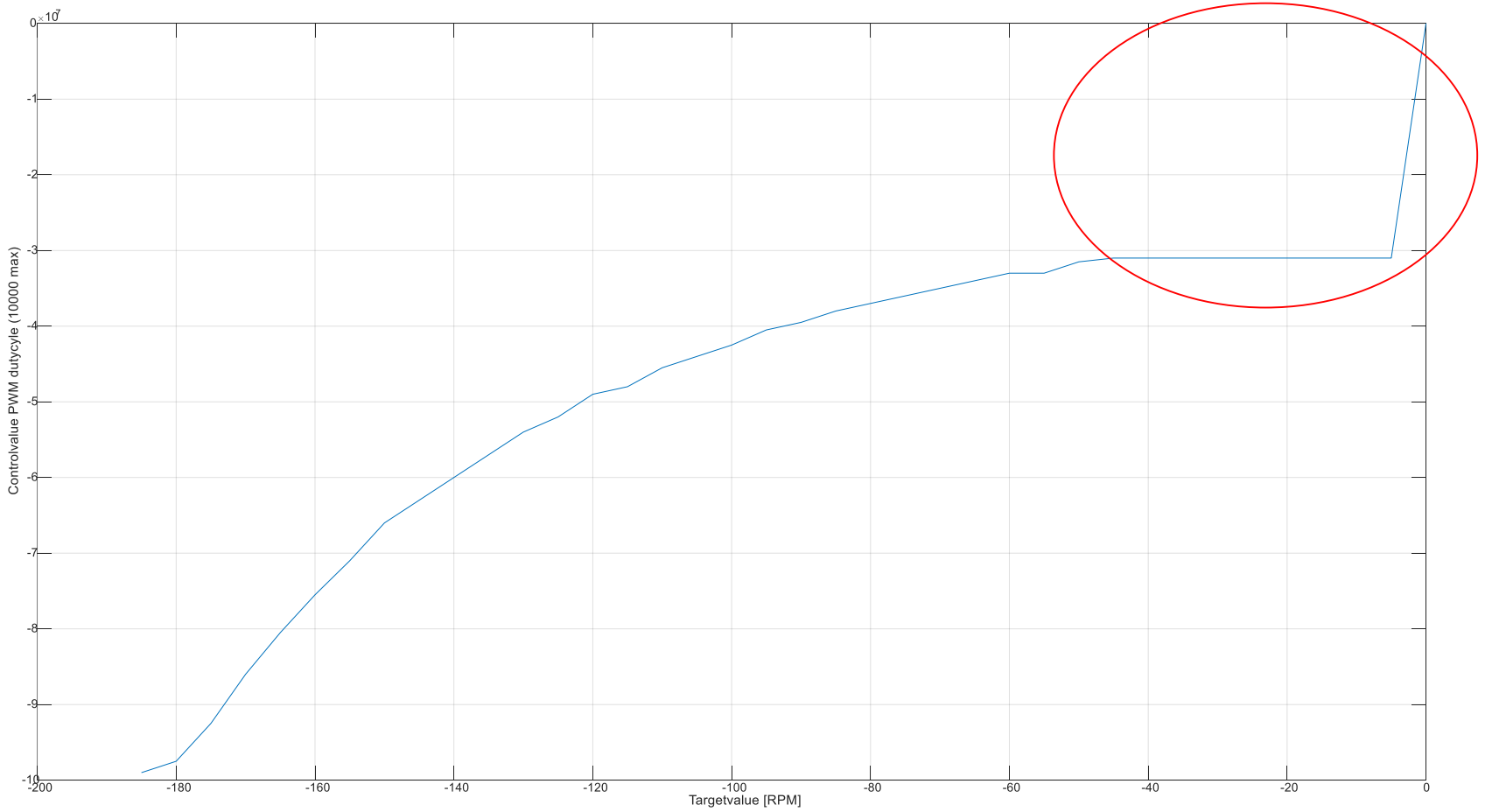
Auto-creation of a lookup table



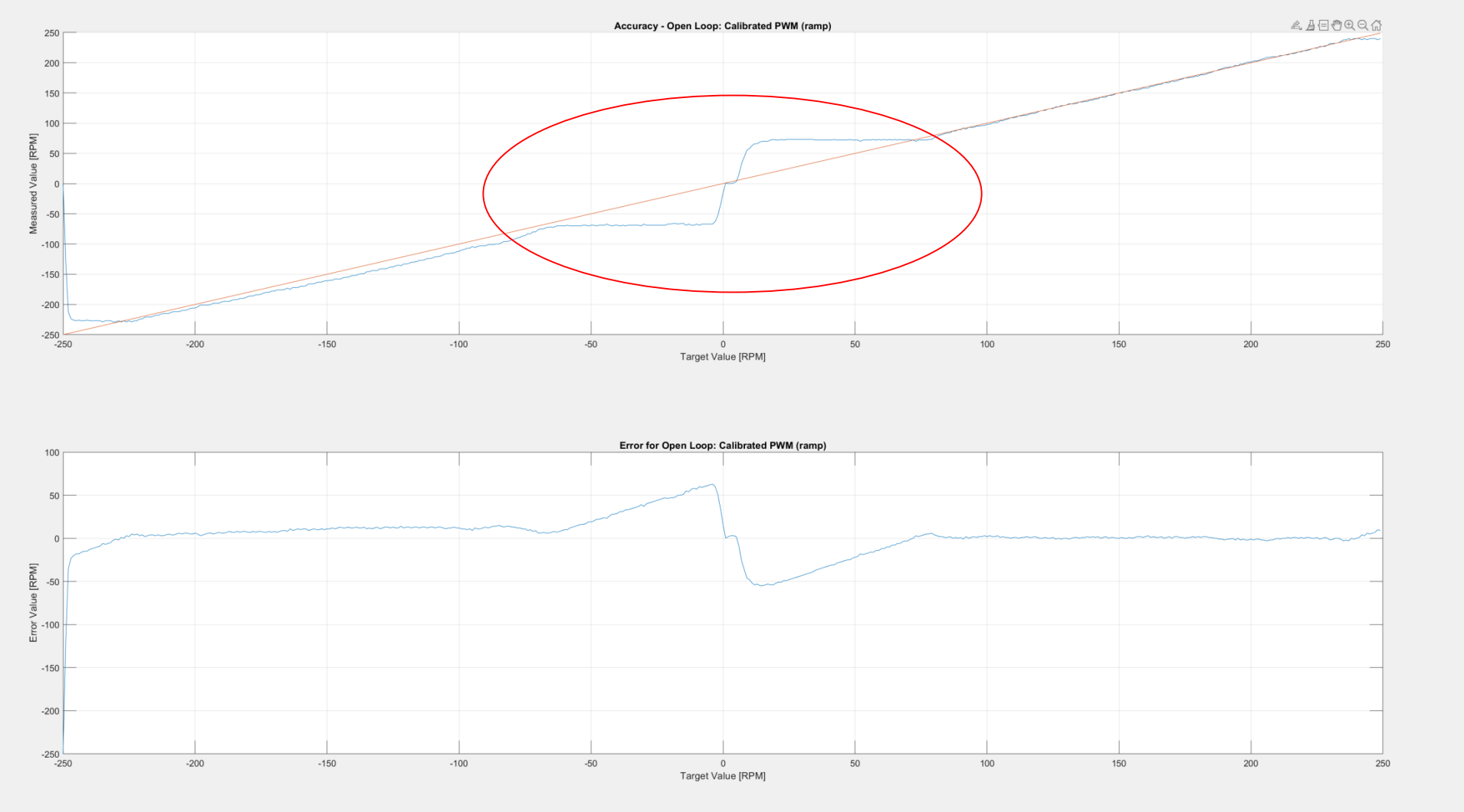
Calibration Curve Forward (0…195 RPM)



Calibration Curve Reverse (0…-195 RPM)



Resulting speed curve using the lookup table



Evaluation Open Loop - Calibration Curve

Open loop is a linear Approach.

- ~~Way better behavior compared to linear approach~~
- Efforts for creating are doable.
- Works without sensor ✓
  (although you need a sensor to create this calibration curve)
  → Theoretically, we can mount a decoder on the axis use that for calibration and use the engines without the decoder in operational mode.

- Does not consider “environmental distortions”, e.g. mechanical engine load, battery voltage
- Strong impact of engine parameters on implementation

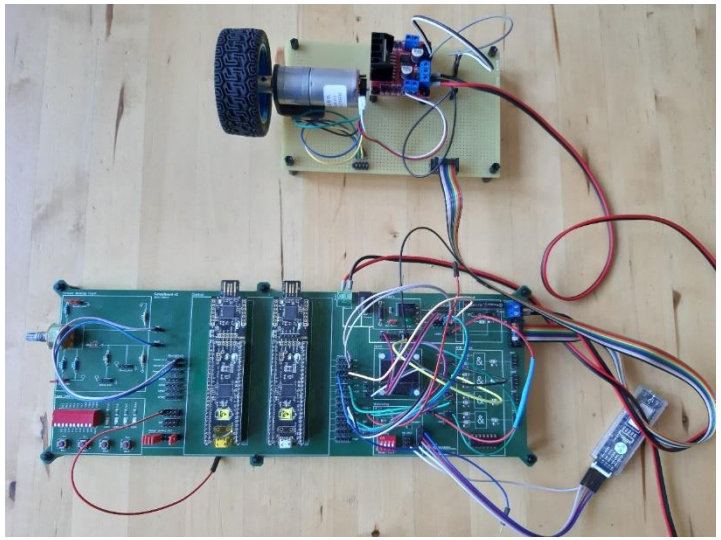
→ If one engine has a higher load, it will slow down. The controller doesn't adjust that.
→ When Battery voltage is going down, we will have distortions.
→ Whenever the engines are slightly different, we have to create engine calibration curve. (big disadvantage)

A short war story- in a first release….

After creating the lookup table, the code was integrated into the car project…

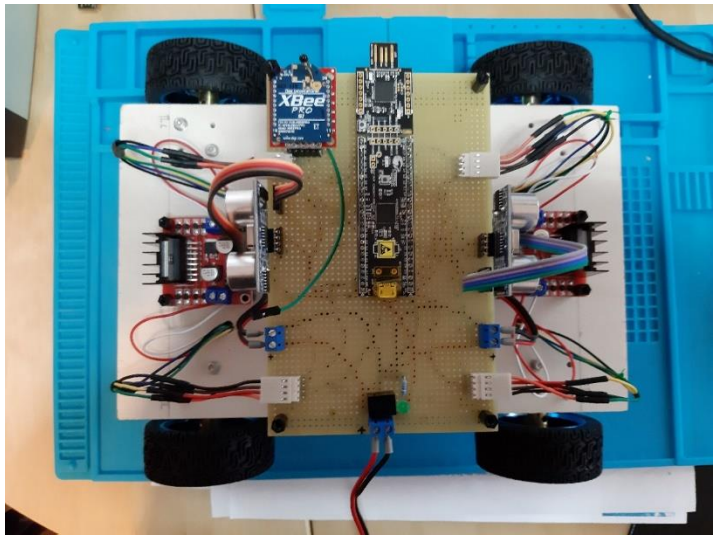
… and an error of around 50 RPM (approx. 25%) was seen!

How it went….



Creation of calibration curve on standalone system,
as the car was not available in the early project stage.

Porting to car platform



Assuming a software bug - Possible buggy sections

	Target Value	Control Value	Measured Value
Hterm sprintf	150rpm	8000	200rpm
Debug	150rpm	8000	520rpm

Rebuilding the system

- Verified the time base with a scope
- Check the Measured Value at different places (to rule out implicite casts)
- Tried alternatives to sprintf
- Simplified the code as much as possible (don’t forget to create a zip before)

Root Cause

- My software was ok (of course ;-)

- The engines have been ordered in two branches.
- The second branch had different parameters – not documented (of course ;-)

- The wrong value in the debugger…
- sprintf was ok, verified with atoi
- Local values are not really reliable (in general)
- Although I checked the value at several placed
- Strange…

Got it – maybe….

```
607  //=============================================================================================
608
609
610  /**
611   * Create a callibration curve and dump it over UART
612   * @return RC_SUCCESS if ok, RC_ERROR_CANCELED if finidhed, else error code
613   */
614  RC_t ENGINE__LinearCurve()
615  {
616      ENGINE__counter++; //0..100
617
618      sint16_t targetspeed = (sint16_t)((ENGINE__counter * 4 - 200));
619      sint16_t controlspeed = targetspeed * 50;     // No difference in this case
620      sint16_t measuredspeed = 0;
621
622
623
624      //Read cycletime from current record
625      uint16_t cycletime = ENGINE__getCycleTime(ENGINE_ID);
626
627      ENG_SetPWM(ENGINE_ID, controlspeed);
628      ENG_GetSpeed(ENGINE_ID, cycletime, &measuredspeed);
629
630      ENGINE__UartWriteRecord(targetspeed, measuredspeed, co
631
632      return RC_SUCCESS;
633
634  }
635
636  /**
```

Locals

Name	Value	Address	Type	Radix
targetspeed	-88		short	decimal
controlspeed	-4400		short	decimal
measuredspeed	-327	0x20007EFA (All)	short	decimal
cycletime	0x0064		unsigned short	Default

HTerm 0.8.1beta

File Options View Help

Disconnect Port COM12 R Baud 115200 Data

Rx 415365 Reset Tx 43 Reset Count 0

Clear received Ascii Hex Dec Bin Save output Clear a

Sequence Overview

Received Data

```
1    5    10   15   20   25   30   35
R2200 -112 -181 -5600
R2300 -108 -178 -5400
R2400 -104 -173 -5200
R2500 -100 -172 -5000
R2600 -96  -167 -4800
R2700 -92  -164 -4600
```

Selection (-)

Input control

Input options

Clear transmitted Ascii Hex Dec Bin

Type ASC

Transmitted data

```
1    5    10   15   20   25   30   35
mdmddddddddefmaaaaaaamaaaaaaaaaaaaa
```

History -/10/10

The debugger has a pretty long latency before stopping. During this time, the engine still runs, incrementing the decoder.

Driving all engines

Settings:

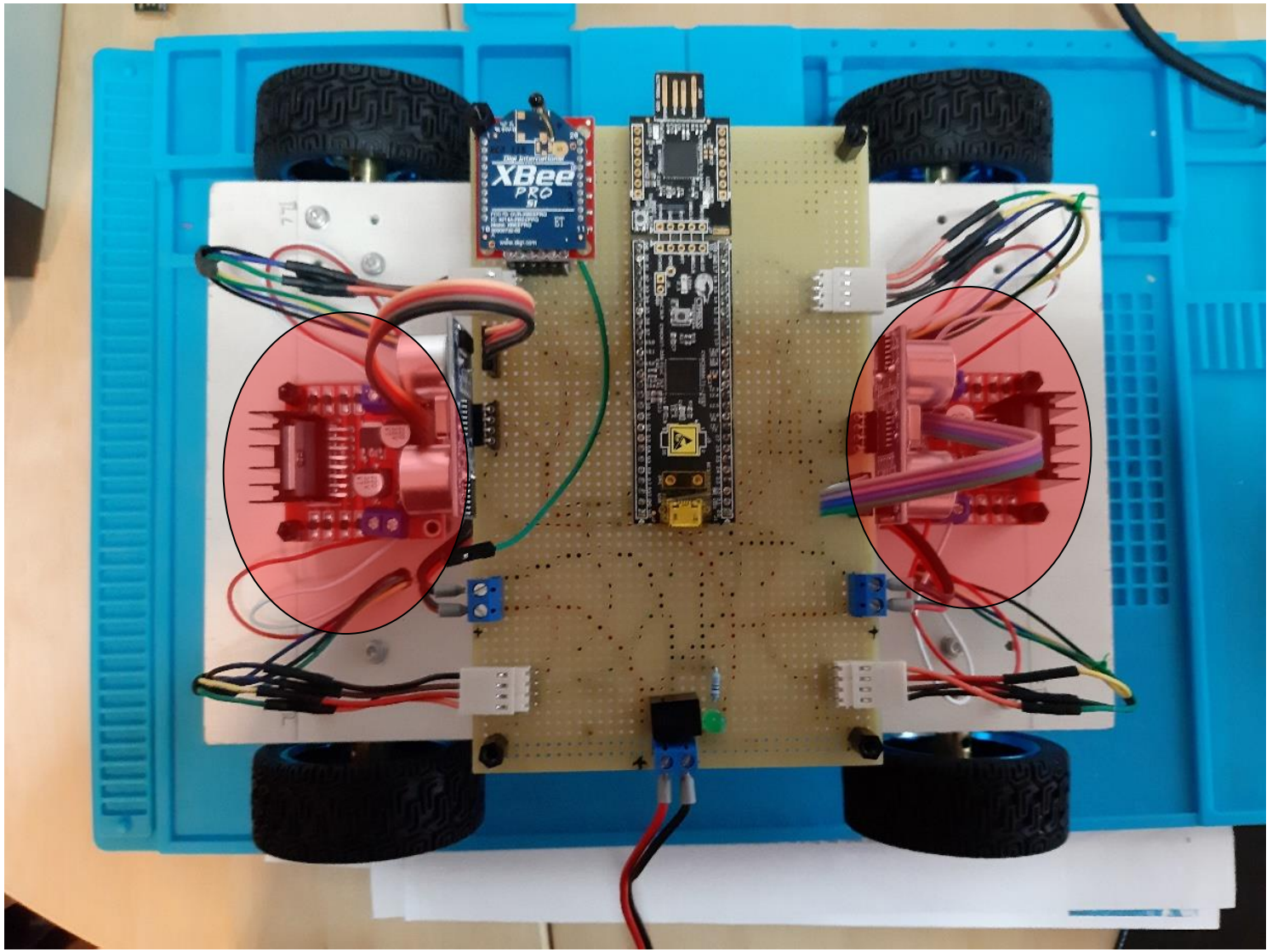
- Control Register is set to 01 for all engines
- PWM is set to 3000 (approx. 30%) for all engines

Observation:

Engine	Rotation	Decoder
Front Left (FL)	Reverse	Reverse
Front Right (FR)	Forward	Reverse
Rear Left (RL)	Forward	Forward
Rear Right (RR)	Reverse	Forward

Left and Right as well as front and rear are inverted!

Inversion of engine and decoder



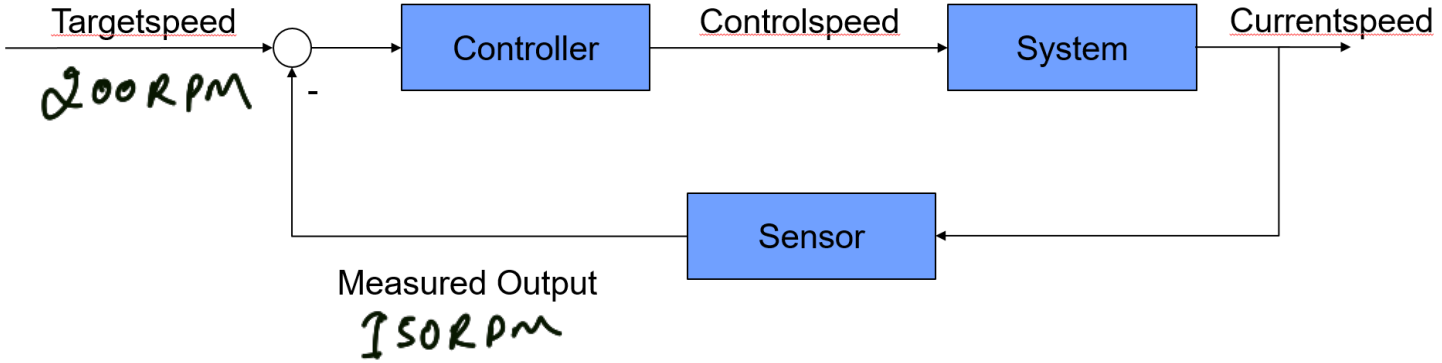
Separation of engine and car configuration

```
/*****************************************************************************/
/* Car Parameters                                                            */
/*****************************************************************************/

struct sEngineOrientation {
    ENG_id_t    m_id;           /**< \brief Identifier of the engine */
    sint8_t     m_dir_eng;      /**< \brief Orientation of the engine (-1 or 1) */
    sint8_t     m_dir_dec;      /**< \brief Orientation of the decoder (-1 or 1) */
};
typedef struct sEngineOrientation ENG_orientation_t;

const ENG_orientation_t ENG_orientation[ENG_ALL] = {
    { ENG_RR,   -1,     -1},    //PWM_Rear_2
    { ENG_RL,    1,      1},    //PWM_Rear_1
    { ENG_FR,    1,     -1},    //PWM_Front_2
    { ENG_FL,   -1,      1},    //PWM_Front_1
};
```

Adding this to the driver, will allow us to drive all engines with respect to the car coordinate system!

PID Controller

The small errors that proportional error doesn't recognise become large and effect the engine logic. So, we have the integral error that helps to identify this over time.



Proportional error: $e_p(k) = v_{target}(k) - v_{measured}(k) = 50RPM$ → An error of 50rpm needs to be corrected.

Integral error: $e_i(k) = (e_i(k-1) + e_p(k)) \times \Delta t$

Differential error: $e_d(k) = \frac{e_p(k) - e_p(k-1)}{\Delta t}$ → there we check, how fast the error is changing. We need to slow the change & make sure that we are not overreacting.

Controlspeed calculation: $v_{control}(k) = K_p \times e_p(k) + K_i \times e_i(k) + K_d \times e_d(k)$

PID Parameter Selection

The challenging part of the closed loop control design is the estimation of the parameters K_p, K_i and K_d as well as the sample time t_s

Trial and Error approach (of a software engineer, not a control engineer):

- Start with K_p = ½ Ratio Controlspeed/Targetspeed (10000 / 200 = 50)
- Increase K_p just before oscillating $K_{p\_critical}$
- Reduce K_p well below $K_{p\_critical}$ (just before the system overreacts)
- This will result in a currentspeed below the targetspeed

- Increase K_i to reach targetspeed

- Compare with Open Loop to see if physical limits are reached

Control Engineer Estimation Approaches

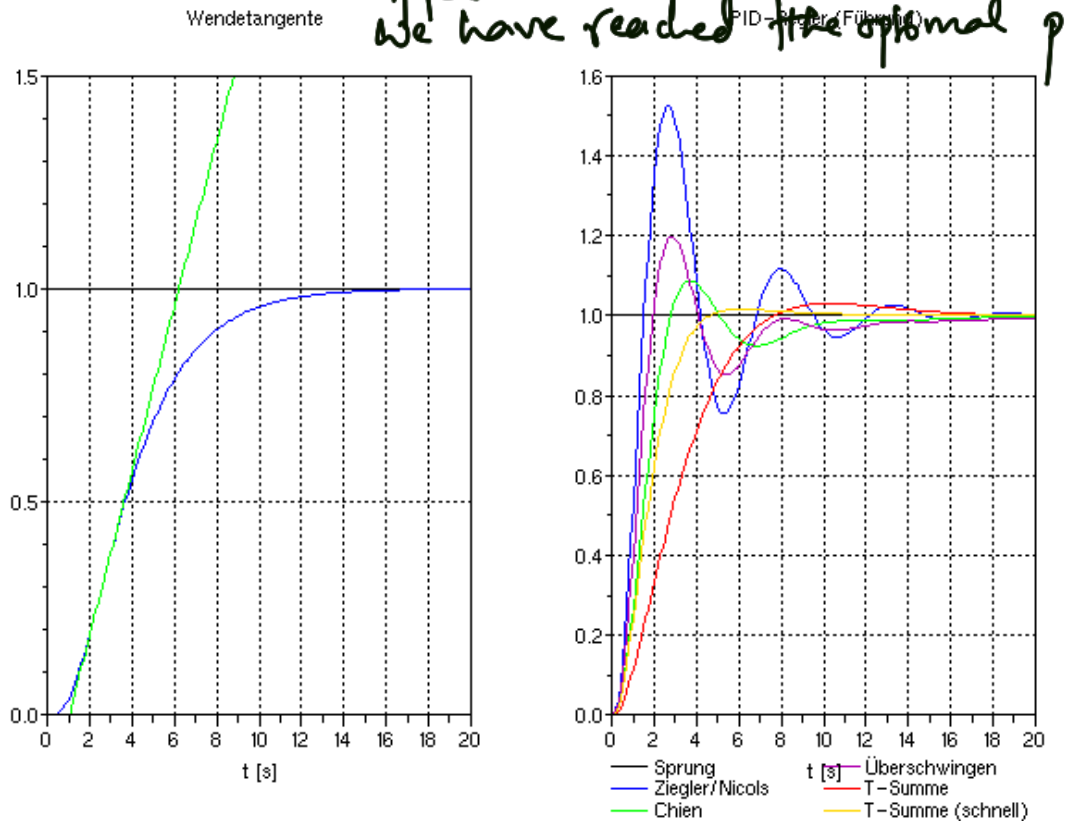
Key idea: When we turn on the engine and read out some parameters from the curve, we can calculate these K_p, K_i, K_d values. → Have to check values for different load conditions, to see if we have reached the optimal point or not.

- Ziegler–Nichols method
- Extension by Chien, Hrones und Reswick (separation guidance and distortion behavior)

Ziegler–Nichols method[1]

Control Type	K_p	T_i	T_d	K_i	K_d
P	$0.5K_u$	–	–	–	–
PI	$0.45K_u$	$T_u/1.2$	–	$0.54K_u/T_u$	–
PD	$0.8K_u$	–	$T_u/8$	–	$K_uT_u/10$
classic PID[2]	$0.6K_u$	$T_u/2$	$T_u/8$	$1.2K_u/T_u$	$3K_uT_u/40$
Pessen Integral Rule[2]	$7K_u/10$	$2T_u/5$	$3T_u/20$	$1.75K_u/T_u$	$21K_uT_u/200$
some overshoot[2]	$K_u/3$	$T_u/2$	$T_u/3$	$0.666K_u/T_u$	$K_uT_u/9$
no overshoot[2]	$K_u/5$	$T_u/2$	$T_u/3$	$(2/5)K_u/T_u$	$K_uT_u/15$

The ultimate gain (K_u) is defined as 1/M, where M = the amplitude ratio, $K_i = K_p/T_i$ and $K_d = K_pT_d$.

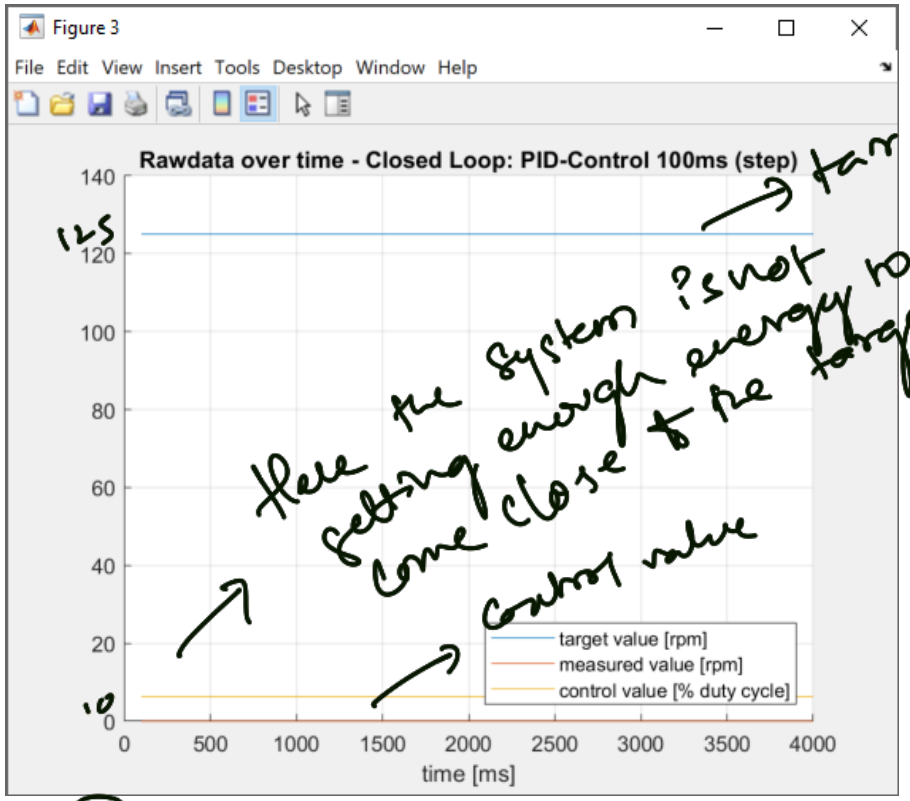


Configuring Kp (1)

Reach around 50% of max target speed of 250 i.e, 125 rpm.

Case(1) : Low Kp Value

Kp = 5, Ki = 0, ts = 100ms



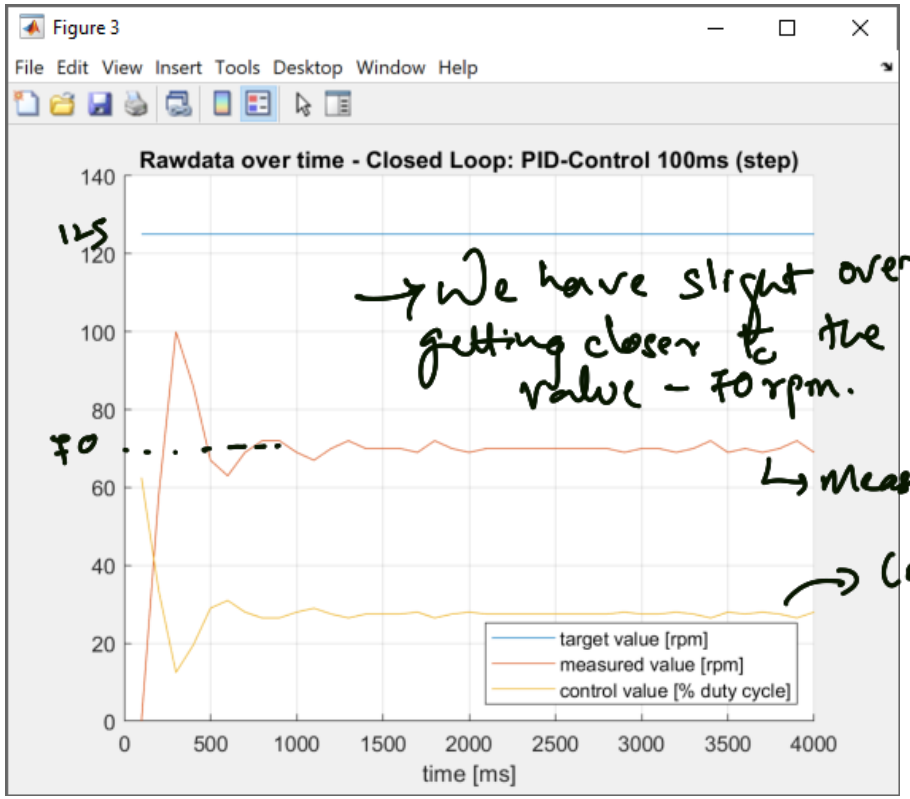
target value

Here the system is not getting enough energy to come close to the target value.

control value

Case(2)

Kp = 50, Ki = 0, ts = 100ms



We have slight over-reaction & getting closer to the half of target value - 70 rpm.

↳ Measured value

Control value.

Targetspeed: Value set by the user

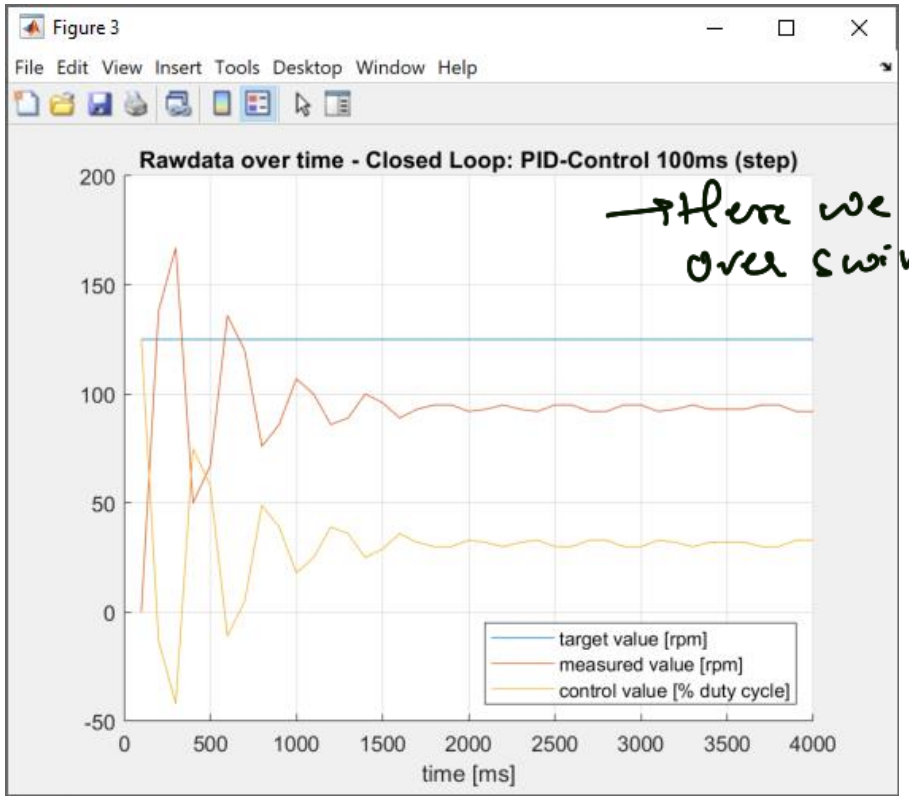
Controlspeed: Speed given to the engine driver

Current Speed: Speed value measured at the engine.

Configuring Kp (2) – Oscillating / Critical range

Case (2)

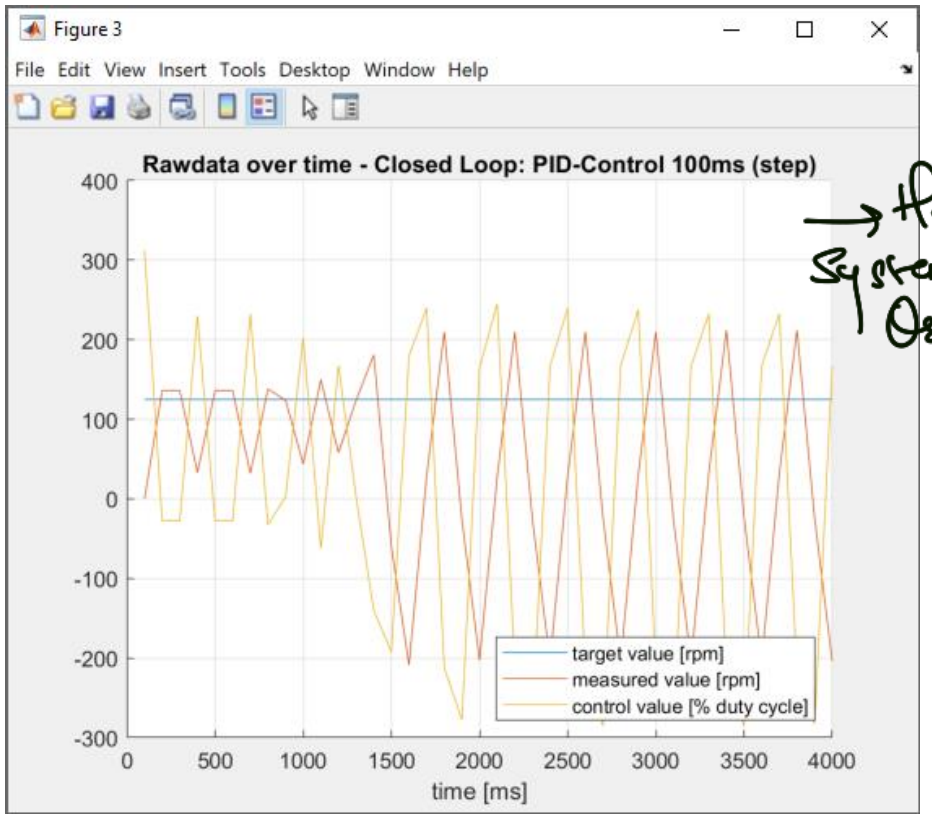
Kp = 100, Ki = 0, ts = 100ms



→ Here we are over swinging.

Case (4)

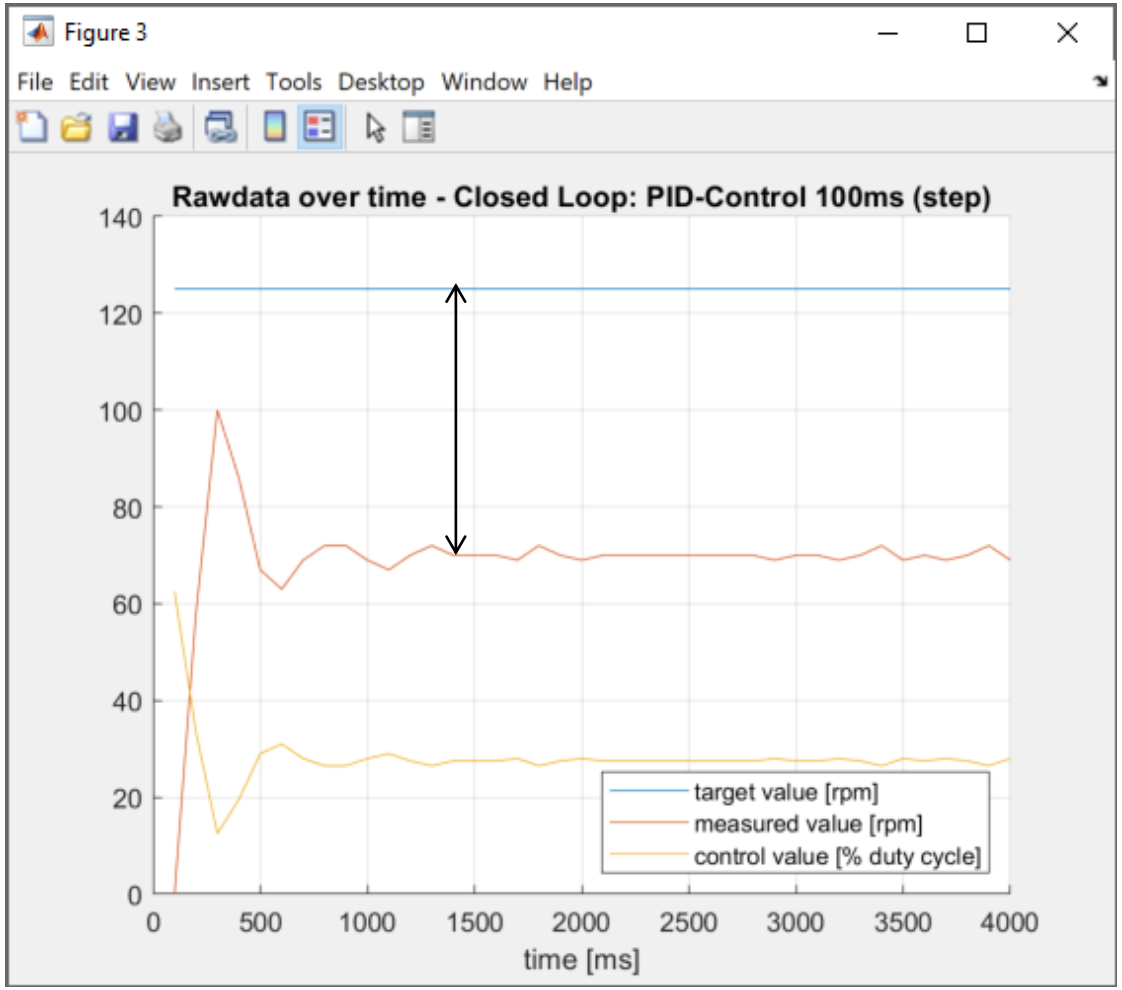
Kp = 250, Ki = 0, ts = 100ms



→ Here the System is Oscillating.

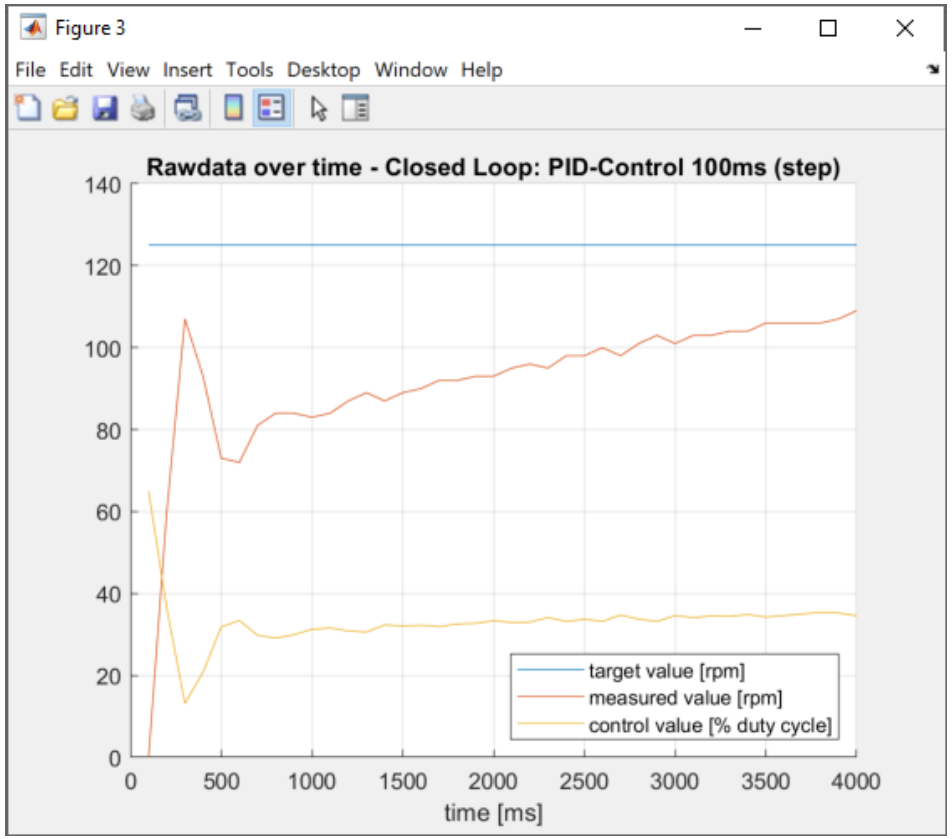
Configuring Ki

- Add a correction value based on the growing (integrating) error

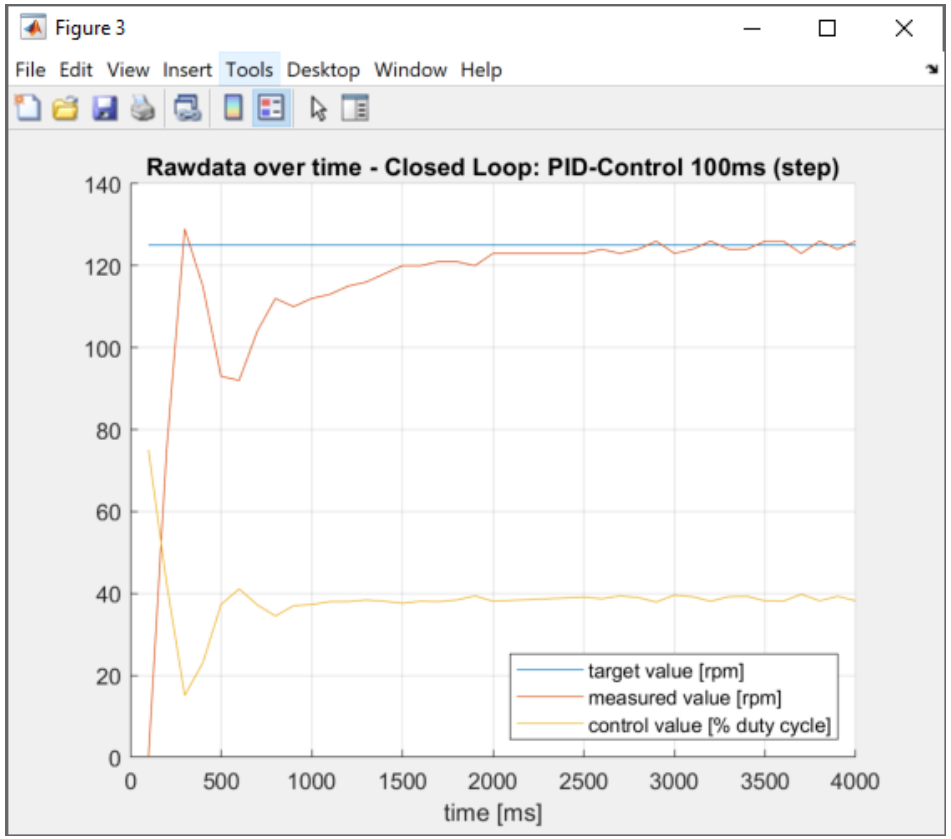


Configuring Ki

Kp = 50, Ki = 20, ts = 100ms

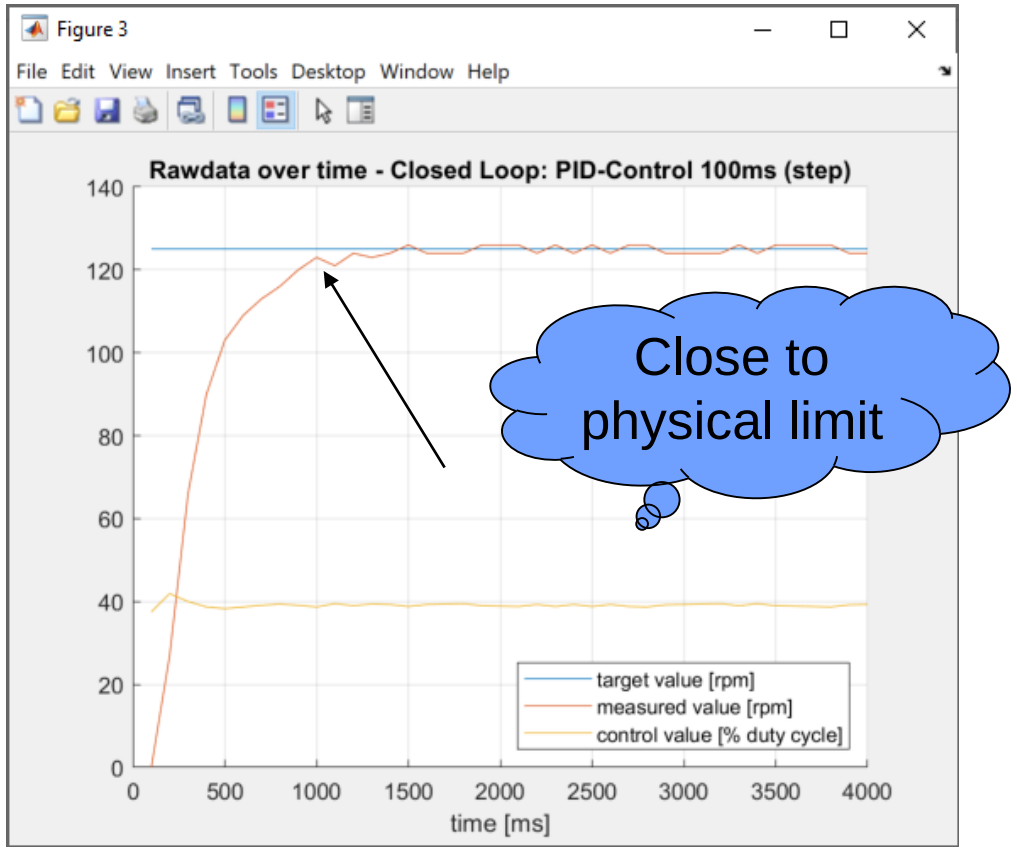


Kp = 50, Ki = 100, ts = 100ms

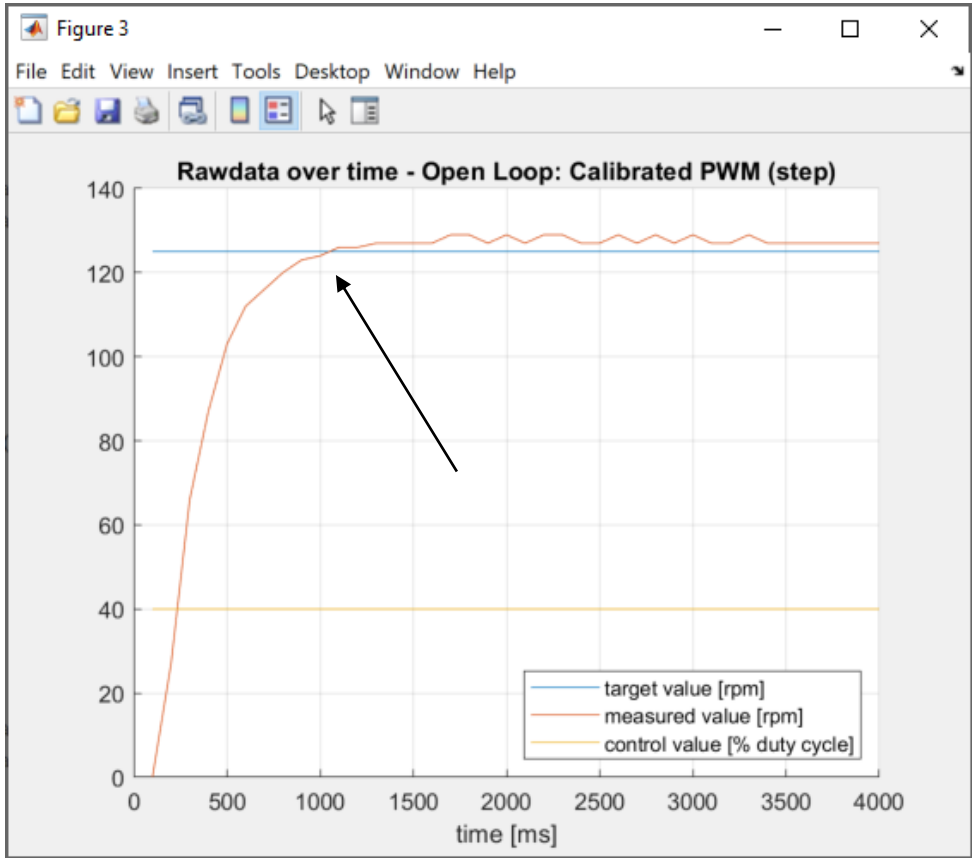


A bit more finetuning and comparison with Open Loop (Step)

Kp = 20, Ki = 100, Kd = 0, t = 100ms

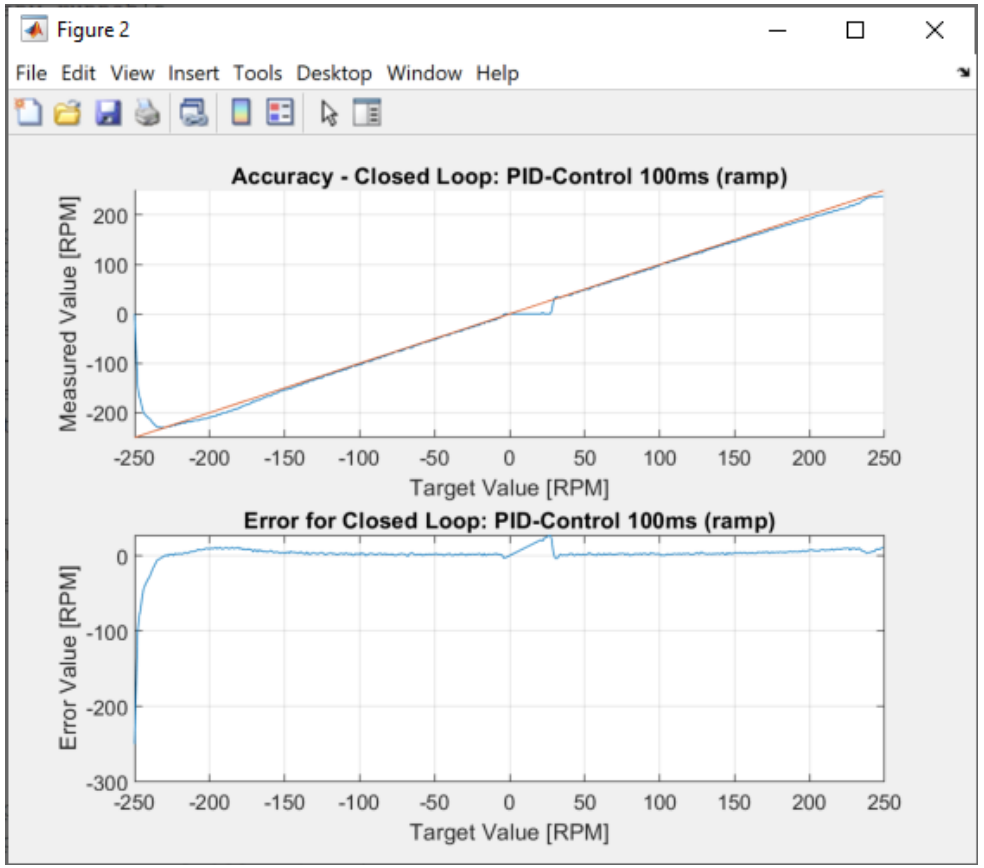


Open Loop

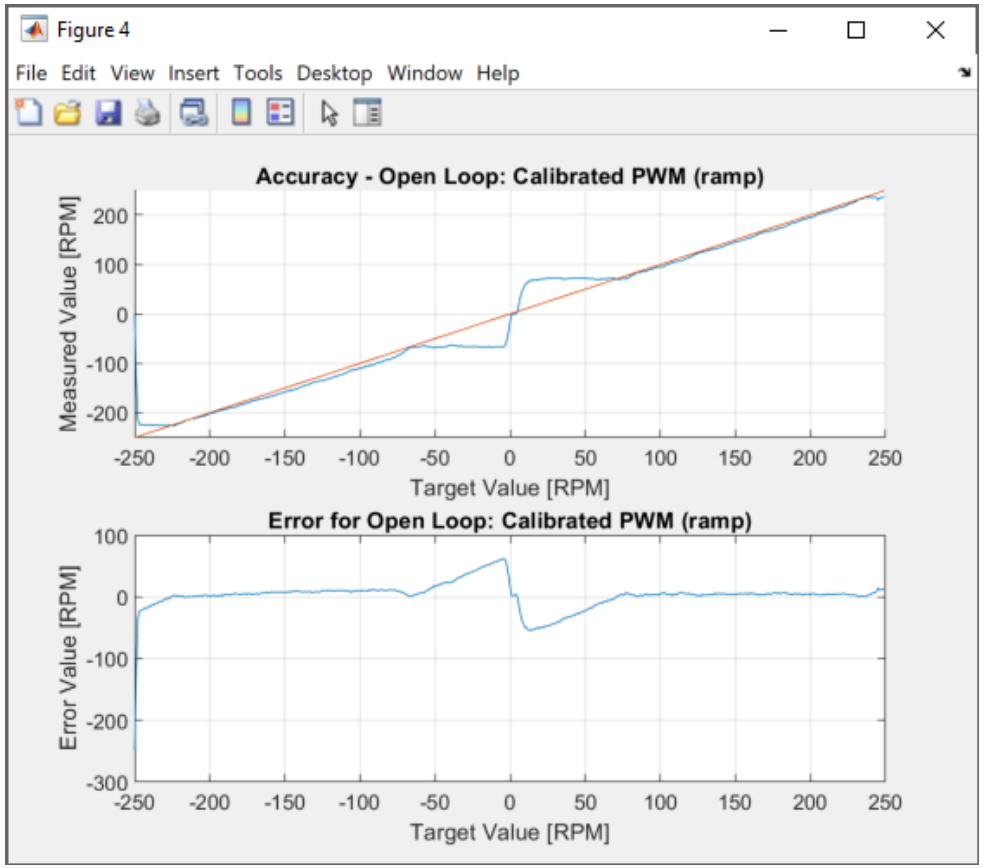


Comparison with Open Loop (ramp)

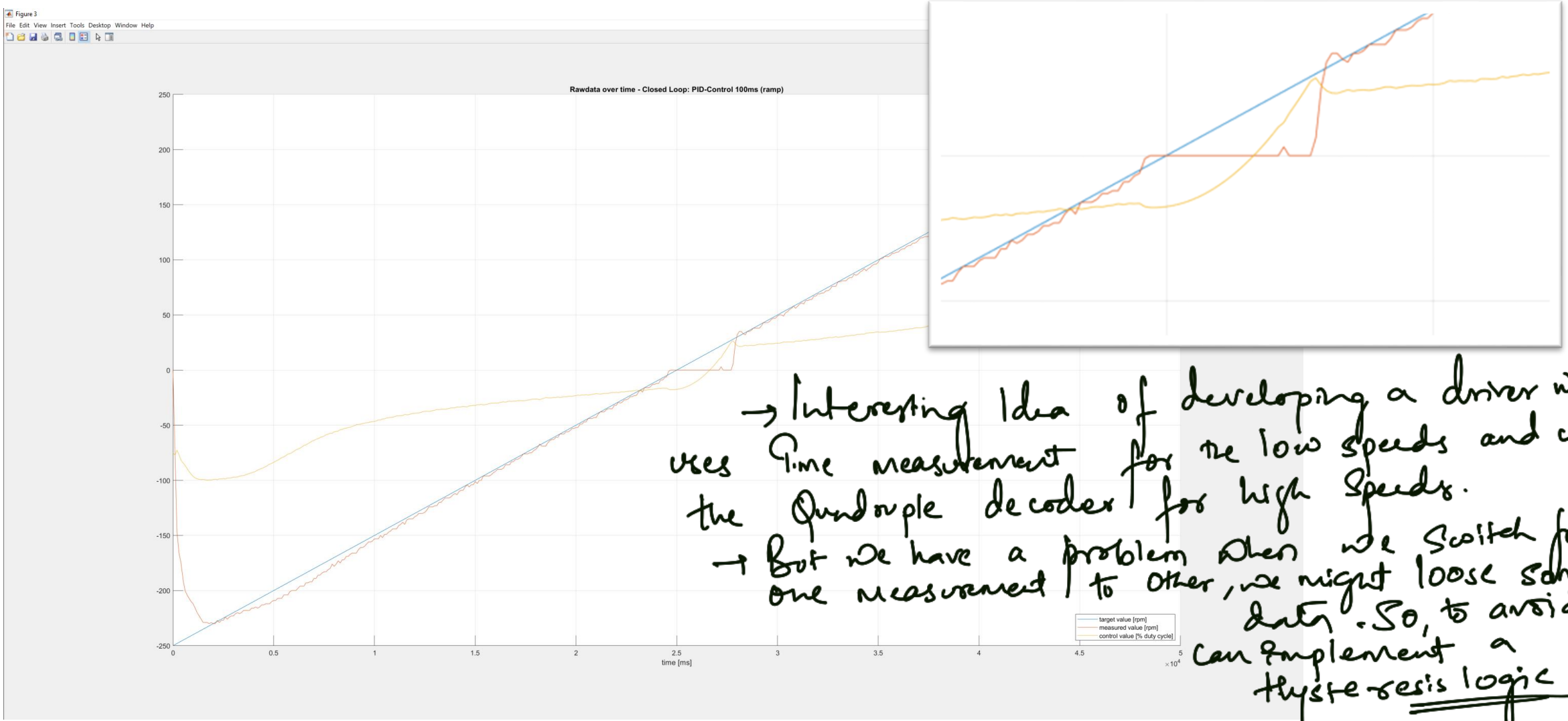
Kp = 20, Ki = 100, Kd = 0, t = 100ms



Open Loop



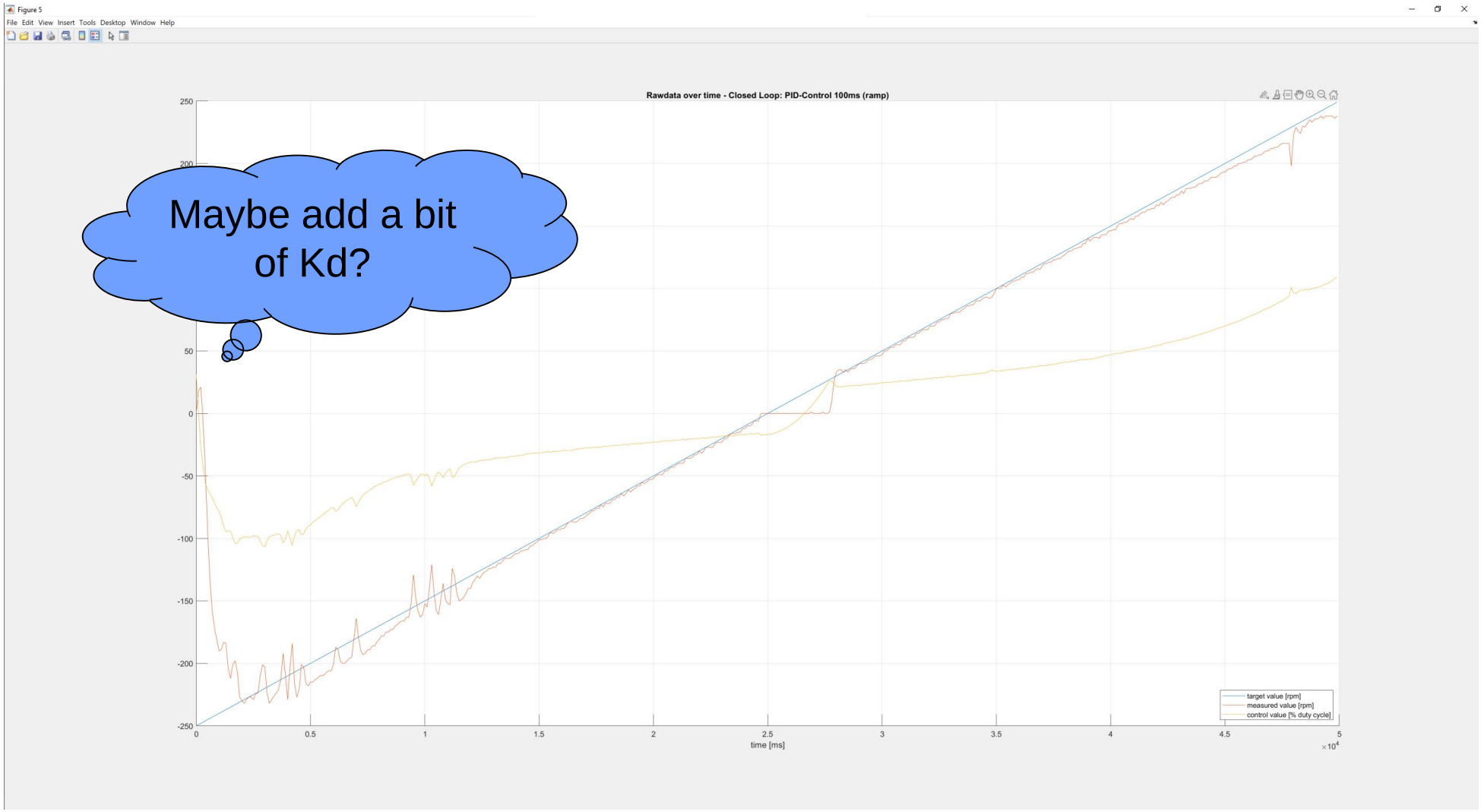
Closed loop ramp – the interesting hysteresis part



→ Interesting Idea of developing a driver which uses Time measurement for the low speeds and use the Quadruple decoder for high Speeds.

→ But we have a problem when we Switch from one measurement to other, we might loose some data. So, to avoid we can implement a hysteresis logic.

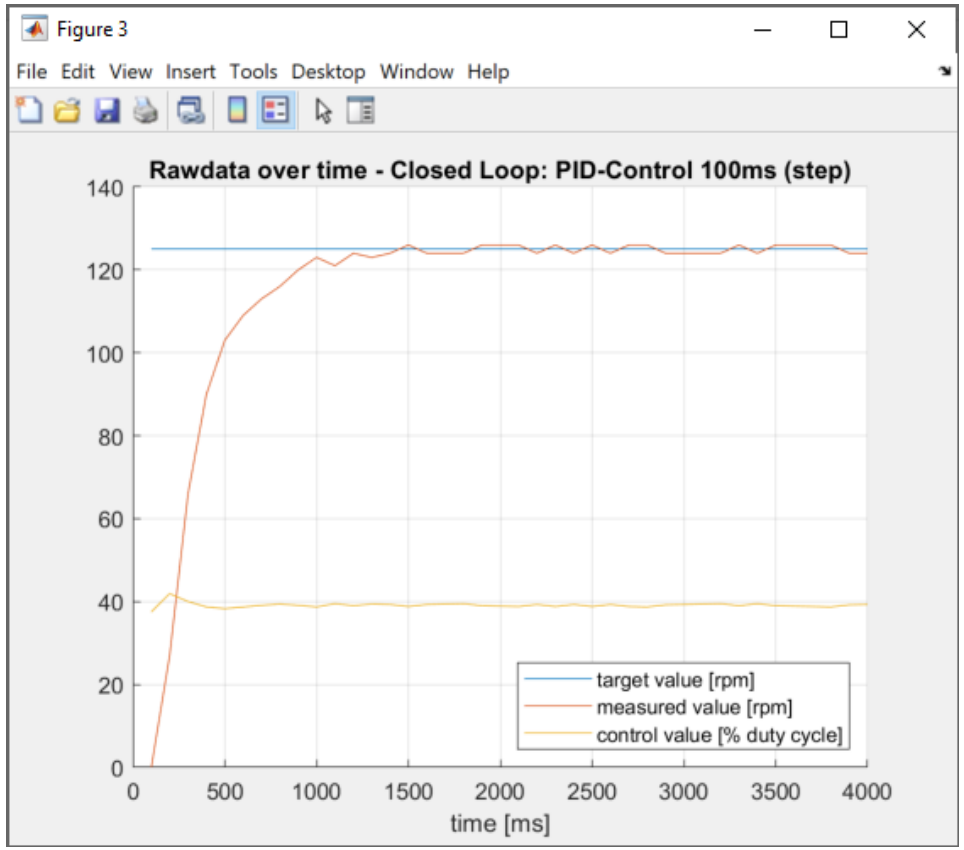
Closed loop ramp – error injection holding torque



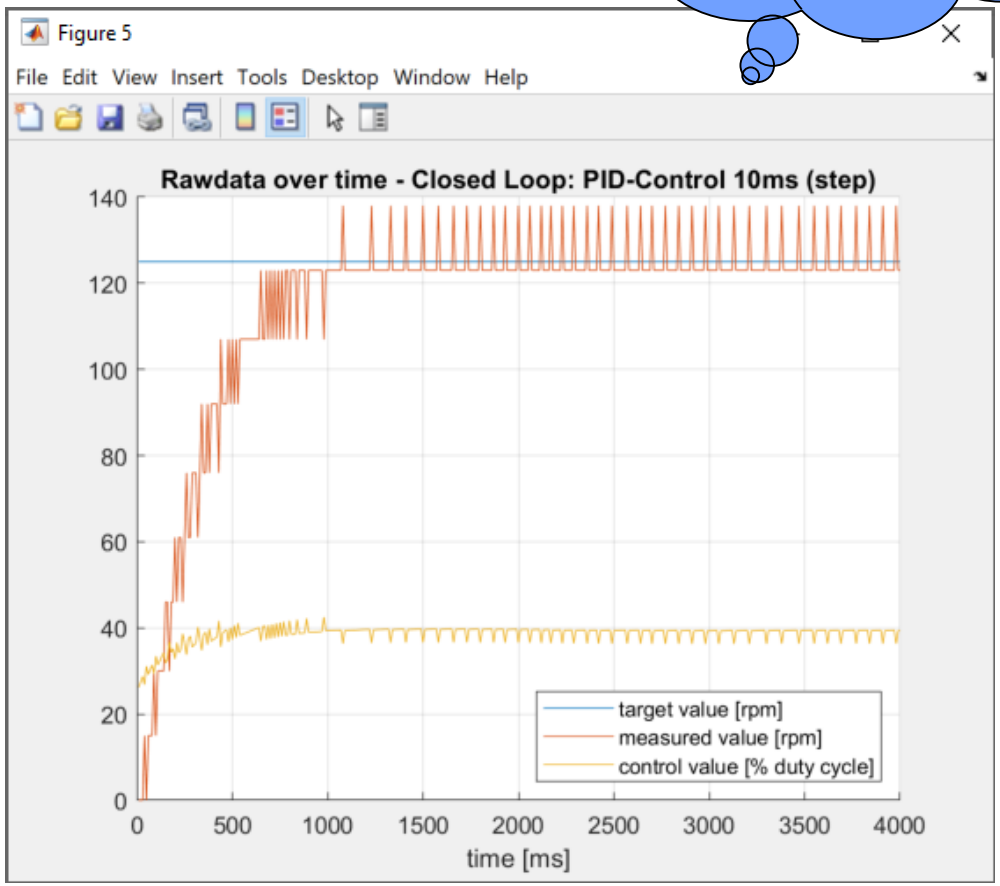
Configuration of the sample time

Precision of the decoder!

Sample Time 100ms



Sample Time 10ms



Precision of the Sensor

We want to use the unit [RPM] as physical unit for the speed:

$$v_{measured} = \frac{\big(decoder(k) - decoder(k-1)\big) * 60 \ * 1000}{\Delta t \ * decoder\_ratio \ * engine\_ratio}$$

- Decoder_ratio = 10
- Engine_ration = 39
- Dt = 10ms

- 1bit decoder ➔ 15.38 rpm

Increasing the ticktime will improve precision but decrease dynamics! Compromise required! Check the mechanical time constant!

Float versus integer math

```
RC_t ENG_SetEngineClosedLoopPID(ENG_id_t id, uint16_t ticktime, sint16_t targetspeed, sint16_t* measuredspeed,
sint16_t* controlspeed)
{

    float32_t error = 0;
    static float32_t integralError = 0;
    static float32_t lastError = 0;
    float32_t differentialError = 0;
    RC_t result;

    //Get the current speed
    result = ENG_GetSpeed(id, ticktime, measuredspeed);

    //Control logic
    error = targetspeed - (*measuredspeed);
    integralError += ((error * ticktime) / 1000);
    differentialError = (error - lastError) * (1000/ticktime);
    lastError = error;

    *controlspeed = (sint16_t)(ENG_CONTROL_KP * error + ENG_CONTROL_KI * integralError +
                    ENG_CONTROL_KD * differentialError);

    result = ENG_SetPWM(id, *controlspeed);

    return RC_SUCCESS;

}
```

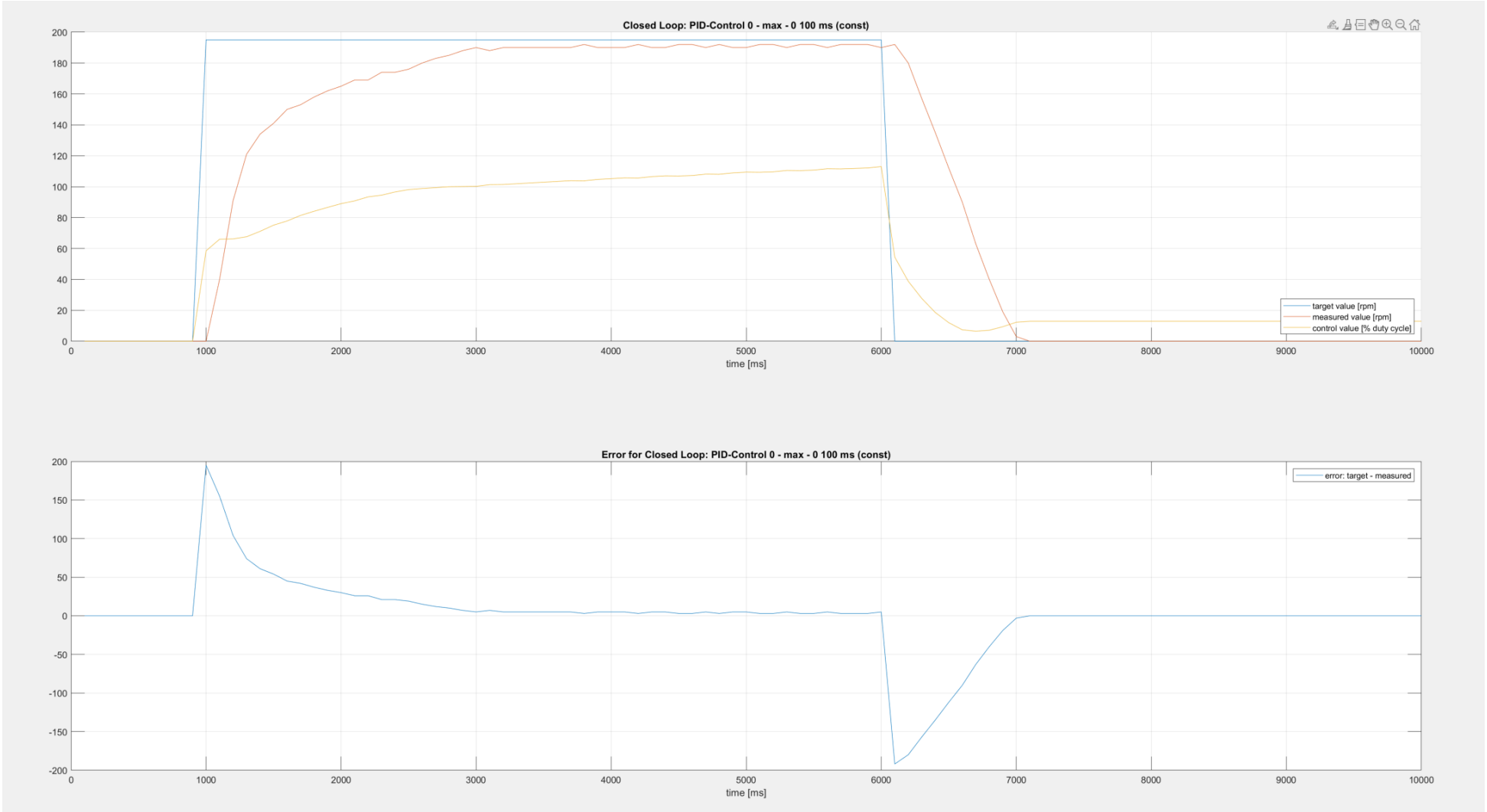
→ We have to consider float values, especially for the error values because they start with small values which are close to 0 and if we are using integers, especially for the integral error, it would be permanently scaled down to 0 & never grow.

→ for proportional Error, maybe integer value could be used

→ We should not mix different types.

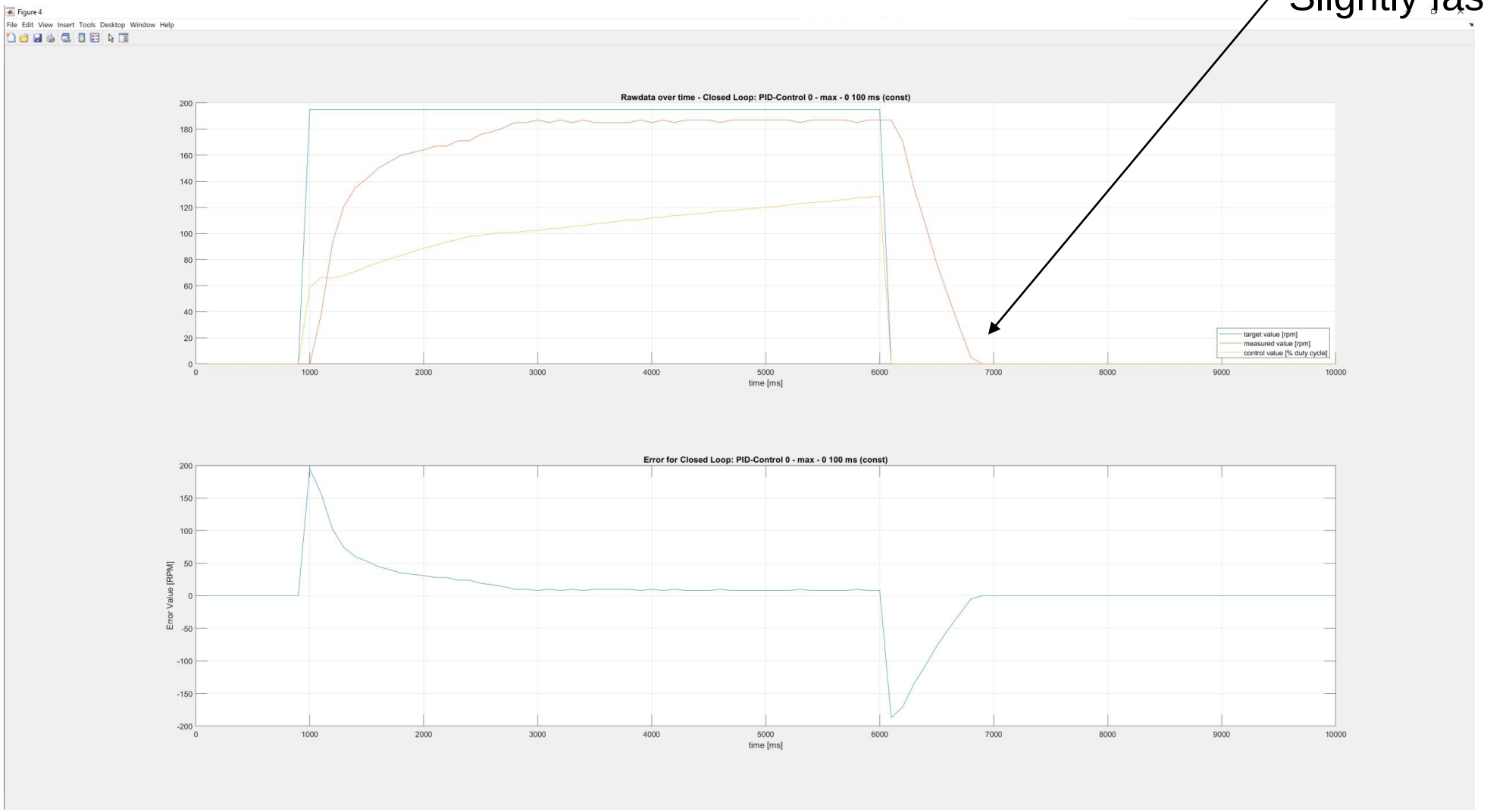
→ In case of value adding to the peripheral, then this float value must be casted to sint16_t.

Resulting Dynamics (controlvalue set by PID)



Resulting Dynamics (Active Brake)

Slightly faster halt



Evaluation Closed Loop - PID

- Better fitting to the targetspeed
- (Partly) independent of engine parameters
- Control of errors

like different mechanics, electric parameters.
We have not analyzed Kd value.

- Efforts for calibration
- Stability / oscillation

WrapUp

- CDD Driver implementation for engine
- Open loop control
  - Linear approach
  - Calibration Curve
- Closed loop control
  - PI calibration
- Comparison of the 2 approaches